

Moldsafe – Aplicație pentru predicția apariției mucegaiului

Candidat: Amalia-Gordana AVRAMOV

Coordonator științific: ș.l.dr.ing. Daniela STĂNESCU

Sesiunea: Iunie 2023

REZUMAT

De-a lungul anilor, tehnologia a devenit o componentă esențială în numeroase domenii, inclusiv în domeniul predicției mușcăiului. Mușcăiul reprezintă o problemă frecventă și persistentă în multe medii interioare, având un impact negativ asupra sănătății umane și a calității aerului. Obiectivul acestei lucrări este monitorizarea temperaturii și a umidității pentru calcularea riscului de mușcăi și accesarea acestora în timp real prin intermediul unei interfețe intuitive și prietenoase.

În prima parte a lucrării, este scos în evidență contextul și punctul de plecare al aplicației, alături de o descriere succintă a funcționalităților și a obiectivelor acesteia. Prin urmare, se oferă o imagine de ansamblu asupra importanței predicției mușcăiului și a nevoii de a avea o soluție eficientă în identificarea și monitorizarea acestuia.

Următoarea parte a lucrării se concentrează pe analiza domeniului, în care se prezintă modelul de predicție utilizat și cercetările curente în domeniu. De asemenea, se realizează o comparație între aplicația propusă și alte aplicații similare existente pe piață.

În continuare, sunt prezentate detaliile legate de componentele hardware utilizate în cadrul aplicației. Se prezintă specificațiile tehnice ale microchipului ESP8266, care este folosit pentru conectarea și transferul datelor între dispozitive, și senzorul BME280, care măsoară umiditatea, temperatura și presiunea aerului din încăperea. Aceste componente joacă un rol esențial în funcționarea aplicației, dar pot fi înlocuite cu orice dispozitiv care se poate conecta la MQTT. Urmează o secțiune dedicată tehnologiilor folosite, explicând rolul și importanța fiecăreia. În această parte se pune accentul pe protocolul de comunicare utilizat pentru transmiterea datelor între dispozitive, bazele de date utilizate pentru stocarea și gestionarea datelor colectate de aplicație și tehnologiile folosite pentru server și interfața cu utilizatorul.

Cu toate aceste informații bine fundamentate se face trecerea la partea de proiectare și implementare a aplicației. În această etapă, se descrie arhitectura generală a aplicației, precum și proiectarea hardware și software necesară pentru funcționarea acesteia. Partea de implementarea funcționalităților cuprinde configurarea infrastructurii necesare dezvoltării în Docker, dezvoltarea modulelor ESP8266 și BME280, dezvoltarea componentelor backend și frontend, iar în final integrarea și testarea tuturor împreună.

Lucrarea continuă cu o secțiune dedicată utilizării aplicației, descriind interfețele și funcționalitățile oferite de aplicație, precum procesul de înregistrare, dashboard-ul aplicației și modul de vizualizarea parametrilor colectați. Tot aici, se regăsește și o secțiune dedicată procesului de testare a aplicației.

Ultima parte a lucrării oferă o evaluare generală a proiectului și a aplicației de predicție a mușcăiului, evidențiind concluziile și explorând posibilitățile de dezvoltare și extindere a aplicației în viitor.

CUPRINS

CUPRINS	3
1. INTRODUCERE.....	5
1.1. CONTEXT.....	5
1.2. DESCRIEREA PROIECTULUI	5
2. ANALIZA DOMENIULUI.....	7
2.1. MODELE DE PREDICȚIE	7
2.2. . APLICAȚII ASEMĂNĂTOARE	9
1. COMPONENTE HARDWARE.....	12
3.1. Modulul ESP8266	12
3.2. Senzorul BME280	13
2. TEHNOLOGII FOLOSITE	15
4.1. MQTT.....	15
4.2. MOSQUITTO	16
4.3. PlatformIO	16
4.4. NestJS	17
4.5. REACT	19
4.6. VITE	23
4.7. BAZE DE DATE	24
4.8. DOCKER	25
4.9. DEVELOPMENT CONTAINER	26
4.10. PRETTIER.....	26
4.11. GIT	27
3. PROIECTAREA SISTEMULUI.....	28
5.1. ARHITECTURA SISTEMULUI	28
5.2. PROIECTAREA HARDWARE	29
5.3. PROIECTAREA SOFTWARE	30
4. IMPLEMENTAREA FUNCȚIONALITĂȚILOR	33
6.1. CONFIGURARE CONTAINER DOCKER	33
6.2. CONFIGURARE DEVELOPMENT CONTAINER	34

6.3. ÎNREGISTRAREA ÎN APLICAȚIE	35
6.4. ESP8266 SERVER	36
6.5. CONECTAREA UNUI CLIENT MQTT	38
6.6. ADĂUGAREA UNUI SENZOR	39
6.7. SCRIEREA ȘI SALVAREA VALORILOR PARAMETRIILOR	40
6.8. CALCULAREA INDICELUI DE MUCEGAI	41
6.9. NOTIFICĂRI	43
5. TESTAREA ȘI UTILIZAREA APLICAȚIEI	45
7.1. PAGINA DE START	45
7.2. ÎNREGISTRAREA ÎN APLICAȚIE	45
7.3. DASHBOARD-UL APLICAȚIEI	46
7.4. ADĂUGAREA UNUI SENZOR	47
7.5. VIZUALIZAREA PARAMETRIILOR	48
7.6. NOTIFICĂRI	49
7.7. TESTAREA MANUALĂ	49
6. CONCLUZII	51
7. DIRECȚII VIITOARE	52
BIBLIOGRAFIE.....	53

1. INTRODUCERE

1.1. CONTEXT

„Prevenția este mai bună ca tratamentul.”

„Tehnologia este arta de a transforma știința în ceva practic.”

Aceste citate celebre au reprezentat o sursă de inspirație pentru crearea unei aplicații de predicție a mucegaiului. Primul citat aparține lui Erasmus și sugerează că este mult mai eficient să previi apariția unei probleme decât să o tratezi după ce s-a produs. În cazul mucegaiului, care poate avea efecte negative asupra sănătății și calității aerului interior, dezvoltarea unei aplicații de predicție poate ajuta utilizatorii să identifice și să prevină apariția acestuia. Prin monitorizarea parametrilor precum temperatura și umiditatea care favorizează dezvoltarea mucegaiului și posibilele măsuri preventive, aplicația poate încuraja utilizatorii să acționeze înainte ca problemele să se agraveze. Cel de-al doilea citat îi aparține lui Marcio Barrios, semnificând că tehnologia nu se limitează doar la cunoștințe teoretice și descoperiri științifice, ci implică procesul de aplicare și implementare a acestora în scopuri practice. Tehnologia se referă la transformarea și adaptarea conceptelor științifice în soluții tangibile, produse și servicii care îmbunătățesc modul în care interacționăm cu lumea și ne desfășurăm activitățile. Este o abordare creativă și practică a utilizării cunoașterii pentru a rezolva probleme și a satisface nevoile umane într-un mod eficient și inovator.

Evoluția tehnologiei a facilitat dezvoltarea acestei aplicații, care aduce beneficii semnificative în ceea ce privește protecția împotriva problemelor de sănătate cauzate de mucegai și promovarea unui mediu de viață mai sigur și mai sănătos.

1.2. DESCRIEREA PROIECTULUI

Ideea principală care a stat la baza dezvoltării aplicației **Moldsafe** a fost integrarea tehnologiilor de ultimă oră și a unor algoritmi de analiză pentru a furniza informații relevante pentru identificarea condițiilor propice dezvoltării mucegaiului, folosind o arhitectură modulară care asigură scalabilitate și potențial de creștere.

Aplicația este concepută pentru a oferi utilizatorilor un acces facil la informațiile despre mediul interior, în special în ceea ce privește parametrii critici de temperatură și umiditate care favorizează dezvoltarea mucegaiului pentru a identifica riscul de formare a mucegaiului într-o anumită încăpere. Aceasta integrează în mod inteligent informațiile de la senzorul de temperatură și umiditate conectat la microcipul ESP8266 pentru a furniza o imagine de ansamblu completă și detaliată a tuturor camerelor în care sunt adăugați senzori și măsurătorile pentru fiecare în parte.

Un aspect important al aplicației este că permite utilizatorilor să vizualizeze în timp real valorile citite de senzori care sunt stocate într-o bază de date modernă special concepută pentru a salva date de tip serie temporală.

Procesul de adăugare a senzorilor în aplicație este unul destul de ușor. Utilizatorii pot naviga către secțiunea de gestionare a camerelor și senzorilor, unde pot adăuga și configura noi senzori în câțiva pași simpli. Aplicația oferă, de asemenea, opțiunea de a elimina senzori în cazul în care se dorește acest lucru.

Mai mult decât atât, în momentul în care valorile înregistrate depășesc anumite limite, aplicația generează notificări pentru a avertiza utilizatorii despre potențialul risc de formare a mușcăiului. Această funcționalitate permite utilizatorilor să reacționeze prompt și să ia măsurile necesare prevenirii dezvoltării mușcăiului și evitarea problemele de sănătate asociate acestuia.

2. ANALIZA DOMENIULUI

2.1. MODELE DE PREDICȚIE

Există diferite modele de predicție a mucegaiului care pot fi utilizate pentru a evalua și anticipa riscul de dezvoltare a mucegaiului într-un mediu dat. În general, aceste modele se bazează pe analiza datelor referitoare la umiditate, temperatură, calitatea aerului și alți factori relevanți pentru dezvoltarea mucegaiului.

Modelul ales pentru această aplicație este un model matematic care a fost publicat pentru prima dată în anul 1999, într-un articol intitulat „*A model for predicting the growth of mold on various building materials*”. Acest model a fost dezvoltat de Kari Hukka și Hannu Viitanen de la Technical Research Centre of Finland (VTT). Cercetarea a inclus mai multe studii experimentale privind condițiile de creștere a mucegaiului, în primul rând pe lemn, dar și pe alte materiale de construcție. De-a lungul anilor, cercetătorii și specialiștii în domeniul mucegaiului au adus ajustări și actualizări acestui model, având în vedere noile descoperiri științifice și datele experimentale. Aceste modificări au inclus adăugarea de factori suplimentari, îmbunătățirea parametrilor sau rafinarea ecuațiilor modelului pentru a asigura o predictibilitate mai precisă a creșterii mucegaiului în diverse condiții.

În cadrul modelului Hukka-Viitanen, indicele de mucegai are valori între 0 și 6, valori care sunt utilizate pentru a evalua și clasifica nivelul de mucegai în funcție de gravitatea sa. În figura următoare (Fig. 2.1.1), se poate observa semnificația fiecărei valori. Astfel, valoare 0 indică lipsa mucegaiului, valoarea 1 indică o prezență foarte scăzută a mucegaiului, valoarea 3 prezență moderată a mucegaiului, iar cea mai ridicată valoare semnalează infectarea ridicată cu mucegai.

Mould Index	Covered Area [%]	Typical Feature
0	0	No mould growth
1	≤1	Mould growth is very little, only observable under a microscope
2	≤5	Moderate mould growth that can only be observed under a microscope
3	<10	Visually observable mould growth
4	>10 and <50	Visually observable and dense mould growth
5	>50	Prolific growth on the surface
6	100	Heavy and dense growth

Fig. 2.1.1: Valorile indicele de creștere a mucegaiului [1]

Pentru experimente au fost alese materiale de construcție cunoscute, după cum urmează: „placă de molid (cu margini lipite), beton, beton celular, termoizolație poliuretanică (PUR suprafață de hârtie și cu suprafață lustruită), vată de sticlă, vată poliester, și polistiren expandat (EPS)” [1]. Alburnul de pin a fost folosit ca material de referință. Setul de materiale ales nu poate acoperi în întregime toate produsele din grupa de materiale de construcție, dar ajută la încadrarea fiecărui material într-o anumită clasă în funcție de sensibilitatea la mucegai.

În figura de mai jos (Fig. 2.1.2), sunt prezentate cele patru clase ale sensibilității materialelor de construcție la mucegai. Prima clasă este clasa materialelor foarte sensibile.

Aceste materiale pot fi afectate chiar și în condiții de umiditate relativ scăzută și reprezintă un mediu propice pentru dezvoltarea rapidă a mucegaiului. Următoarea clasă este cea a materialelor sensibile. Și aceste materiale pot fi ușor afectate de umezeală și favorizează creșterea și propagarea mucegaiului în condiții de mediu potrivite. O altă clasă este cea mediu rezistentă. În această categorie se află materialele care pot rezista în anumite condiții de umiditate și au o capacitate mai bună de a respinge sau de a inhiba creșterea mucegaiului. Ultima categorie, cea rezistentă, include materialele care pot tolera niveluri moderate de umiditate și au proprietăți care împiedică înmulțirea și răspândirea rapidă a mucegaiului.

Sensitivity Class	Materials
Very Sensitive	Pine sapwood
Sensitive	Glued wooden boards, PUR with paper surface, spruce
Medium Resistant	Concrete, aerated and cellular concrete, glass wool, polyester wool
Resistant	PUR with polished surface

Fig. 2.1.2: Clasele de sensibilitate pentru diferite materiale [2]

Pentru calcularea indicelui de mucegai este necesar să fie cunoscută umiditatea relativă, temperatura și materialul din care sunt construiți pereții încăperii care se dorește a fi monitorizată.

Primul pas îl reprezintă calcularea umidității relative critice. Aceasta se referă la nivelul de umiditate în care există un risc semnificativ de dezvoltare și creștere a mucegaiului. Aceasta este valoarea de prag peste care condițiile devin favorabile pentru mucegai și potențial pentru problemele asociate, cum ar fi deteriorarea calității aerului interior și impactul asupra sănătății umane. Pentru calcularea umidității relative critice (RH_{crit}) se folosește următoarea formulă (1):

$$RH_{crit} = \begin{cases} -0.00267 T^3 + 0.160 T^2 - 3.13 T + 100, & T \leq 20 \\ RH_{min}, & T > 20 \end{cases} \quad (1)$$

În continuare, se calculează indicele de mucegai maxim (M_{max}), folosind formula de mai jos (2), unde coeficienții A, B și C au diferite valori în funcție de clasa în care este încadrat materialul de construcție al pereților.

$$M_{max} = A + B * \frac{RH_{crit} - RH}{RH_{crit} - 100} + C * \left(\frac{RH_{crit} - RH}{RH_{crit} - 100} \right)^2 \quad (2)$$

Valorile folosite pentru coeficienții A, B și C, dar și pentru umiditatea relativă minimă sunt prezentate în figura de mai jos (Fig. 2.1.3). Potrivit cercetărilor acestea „sunt ele mai bune aproximări ale experților pe baza diferitelor postprocesări numerice a rezultatelor experimentale” [2].

Sensitivity Class	k_1		$k_2 (M_{max})$			RH_{min}
	$M < 1$	$M \geq 1$	A	B	C	%
Very Sensitive, vs	1	2	1	7	2	80
Sensitive, s	0.578	0.386	0.3	6	1	80
Medium Resistant, mr	0.072	0.097	0	5	1.5	85
Resistant, r	0.033	0.014	0	3	1	85

Fig. 2.1.3: Valorile parametrilor pentru clasele de sensibilitate [2]

Următorul pas îl reprezintă calcularea coeficientului k_2 care reprezintă „moderarea intensității de creștere”[2] a indicelui de mucegai (M). Pentru acesta se folosește formula (3), unde M_{max} reprezintă indicele de mucegai maxim calculat în pasul anterior.

$$k_2 = \max[1 - \exp[2.3 * (M - M_{max})], 0] \quad (3)$$

Ultimul pas este calcularea indicelui de mucegai M la momentul t (M_t). Acesta este setat inițial cu 0 ($M=0$ la momentul inițial $t=0$) și este calculat în fiecare oră folosind următoarea formulă:

$$M_t = M_{t-1} + \Delta M \quad (4)$$

În formula de mai sus M_t este indicele de mucegai al orei curente, M_{t-1} este indicele de mucegai al orei precedente, iar ΔM reprezintă modificarea indicelui în timp. Dacă rezultatul ecuației devine negativ, în orice moment t, se setează $M_t=0$. Pentru calcularea lui ΔM se folosește formula (5).

$$\Delta M = \frac{k_1 k_2}{168 * \exp(-0.68 \ln T - 13.9 \ln RH + 0.14W + 66.02)} \quad (5)$$

În ecuația (5), k_1 este factorul de intensitate a creșterii mucegaiului, k_2 este „factorul de atenuare al indicelui de mucegai”[2] calculat anterior, iar W este clasa de sensibilitate. T reprezintă temperatura măsurată în ora curentă la fel și umiditatea relativă (RH) este măsurată tot în ora curentă.

2.2. APLICAȚII ASEMĂNĂTOARE

În prezent, piața aplicațiilor care măsoară indicele de mucegai este relativ restrânsă în comparație cu alte domenii de dezvoltare a aplicațiilor. Cu toate acestea, conștientizarea importanței calității aerului interior și a riscurilor asociate mucegaiului a crescut în ultimii ani, ceea ce ar putea contribui la creșterea pieței acestor aplicații. Mai multe persoane și organizații devin interesate de monitorizarea și prevenirea mucegaiului în mediul lor, în special în contextul sănătății și bunăstării. De asemenea, avansul tehnologic și dezvoltarea senzorilor inteligenți au facilitat dezvoltarea de aplicații specializate pentru măsurarea și monitorizarea calității aerului, inclusiv a mucegaiului.

Pentru a compara sistemul dezvoltat au fost alese două aplicații care pun la dispoziție funcționalități asemănătoare cu aplicația **Moldsafe**.

Prima aplicație este **Airthings Wave**. Aplicația funcționează împreună cu dispozitivele de monitorizare a calității aerului produse de Airthings care prin intermediul tehnologiei Bluetooth, permit monitorizarea calității aerului din diferite încăperi și vizualizarea nivelurilor de mușgai, umiditate, temperatura, calitatea aerului și alți parametri importanți. Dispozitivul care oferă informații despre indicele de mușgai este Wave Mini (Fig. 2.2.1). Aplicația **Airthings Wave** oferă grafice istorice pentru a urmări evoluția în timp a nivelurilor de mușgai și alți poluanți, notificări și alerte pentru a informa asupra schimbărilor semnificative în calitatea aerului, precum și sfaturi și recomandări utile pentru menținerea unui mediu sănătos și sigur.



Fig 2.2.1: Dispozitivul Wave Mini [3]

O altă aplicație este **Foobot** care funcționează împreună cu dispozitivul cu același nume. Dispozitivul **Foobot** (Fig 2.2.2) monitorizează și analizează calitatea aerului interior și exterior, concentrându-se în principal pe măsurarea nivelurilor de poluare chimică și particule în aer, precum și pe monitorizarea temperaturii și umidității. Toate aceste informații sunt afișate și accesibile utilizatorilor prin intermediul aplicației mobile **Foobot**. Utilizatorii pot vizualiza datele în timp real, pot accesa istoricul măsurărilor și pot primi notificări și alerte în cazul în care nivelurile de poluare, particulele, temperatura sau umiditatea depășesc anumite limite prestabilite.



Fig.2.2.2: Dispozitivul Foobot [4]

Aplicația dezvoltată în acest proiect, similar cu cele menționate mai devreme, pune la dispoziție un dispozitiv care măsoară temperatura, umiditatea, altitudinea și presiunea atmosferică. Însă, ceea ce diferențiază această aplicație de celelalte este faptul că acest dispozitiv care măsoară parametrii doriți nu este unic. În aplicație se poate integra perfect orice senzor care se poate conecta la serverul MQTT. De asemenea, aplicația este de tip Progressive Web App (PWA), însemnând că a fost dezvoltată utilizând tehnologii web, dar se comportă ca o aplicație mobilă.

În tabelul 1, se poate observa o analiză comparativă între cele două aplicații **Airthings Wave** și **Foobot** și aplicația **Moldsafe**. Funcționalitățile care au fost analizate și se regăsesc într-o aplicație sunt notate cu ✓ iar cele absente cu X.

Caracteristici și funcționalități	Airthings Wave	Foobot	Moldsafe
Sistem de operare	Android, iOS	Android, iOS	PWA
Magazin de aplicații	Google Play, App store	Google Play, App store	X
Număr de instalări	100K+	5K+	X
Notă	2.6/5(App store)	2.3/5 (App store)	X
Autentificare/Înregistrare	✓	✓	✓
Conectivitate wireless	X	✓	✓
Conectivitate bluetooth	✓	X	X
Măsurarea umidității și temperaturii	✓	✓	✓
Monitorizarea calității aerului	✓	✓	X
Monitorizarea nivelului de radon	✓	X	X
Monitorizare indicelui de mușgai	✓	X	✓
Monitorizare interioară și exterioară	X	✓	X
Notificări/alerte	✓	✓	✓

Tabelul 1: Analiză funcționalități

1. COMPONENTE HARDWARE

3.1. Modulul ESP8266

Modulul NodeMCU ESP8266 a devenit foarte popular în domeniul IoT (Internet of Things) și este utilizat într-o gamă largă de aplicații, inclusiv automatizarea locuințelor, monitorizarea mediului, sistemele de securitate și multe altele.

Modulul NodeMCU ESP8266 este o placă de dezvoltare bazată pe chipsetul ESP8266, care oferă funcționalitate Wi-Fi și capacitate de programare. Acest modul este popular datorită performanțelor sale ridicate și a ușurinței de utilizare.

Acesta vine echipat cu un procesor RISC de 32 bit și 16 bit de instrucțiuni, capabil să gestioneze sarcini complexe. În ceea ce privește memoria, codul trebuie stocat în memoria flash serială care permite scrierea și citirea în grupuri mari de octeți învecinați în spațiul de adrese. De asemenea, oferă conectivitate Wi-Fi, permițând comunicarea cu alte dispozitive și servicii prin rețelele Wi-Fi existente. Astfel, permite interacțiunea cu senzori, actuatori și alte componente IoT, dar și trimiterea datelor către platforme cloud sau aplicații mobile.

O caracteristică importantă este că modulul dispune de un set de pini GPIO (General Purpose Input/Output). În articolul „*Internet of Things and Nodemcu*” se descrie modulul ca având „capabilități puternice de procesare și stocare la bord, care îi permit să fie integrat cu dispozitivele specifice senzorilor prin intermediul GPIO-urilor sale”[5]. Acești pini sunt în număr de 17 (Fig.3.1) și pot fi configurați și utilizați pentru a realiza diverse funcționalități, cum ar fi citirea datelor de la senzori sau controlul dispozitivelor externe.

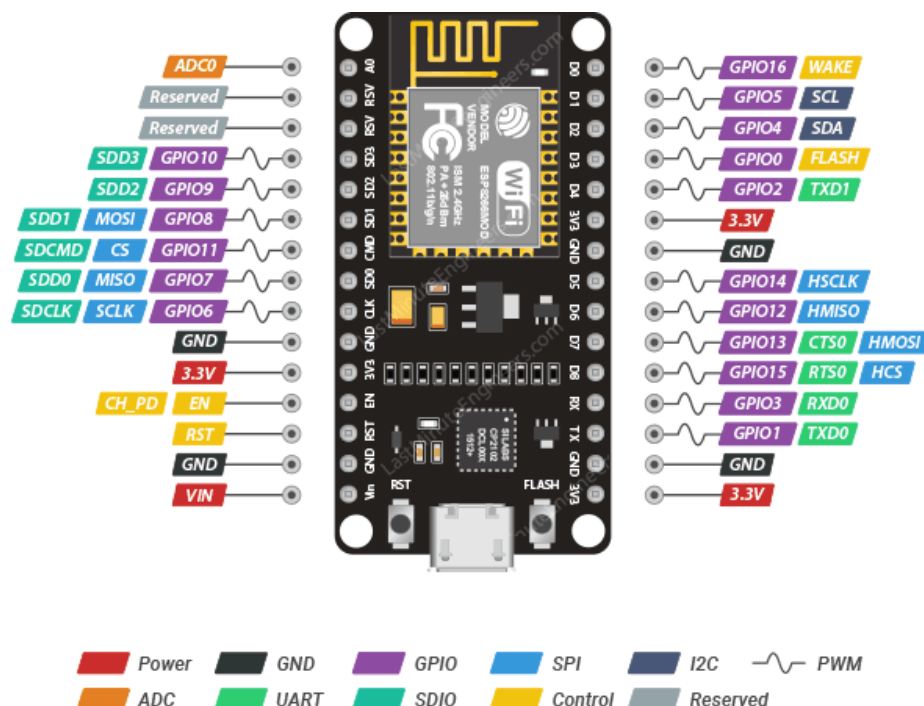


Fig. 3.1: Modulul ESP8266 NodeMCU [6]

Modulul Wi-Fi ESP8266 oferă două moduri de funcționare principale: modul AP (Access Point) și modul STA (Wi-Fi Station). De asemenea, acesta are capacitatea de a comuta între cele două în funcție de necesitățile proiectului.

Modulul ESP8266 în modul AP (Access Point) „acționează ca un punct de acces, astfel încât alte dispozitive să se poată conecta” [7]. Modulul devine astfel un punct de acces Wi-Fi independent și poate furniza o rețea locală la care alte dispozitive se pot conecta. Această funcționalitate este utilă în scenariile în care se dorește crearea unei rețele locale personalizate sau conectarea dispozitivelor Wi-Fi din jur.

Modulul ESP8266 în modul STA (Station) poate funcționa ca o stație Wi-Fi și se poate conecta la alte rețele Wi-Fi existente. Prin intermediul modului STA, modulul ESP8266 poate obține acces la internet sau la alte dispozitive conectate la rețeaua Wi-Fi existentă. Acest mod este folosit în principal atunci când se dorește conectarea modulului la o rețea Wi-Fi existentă pentru a accesa resurse externe sau pentru a comunica cu alte dispozitive prin intermediul rețelei.

Un avantaj al acestuia este că poate fi programat ca orice alt microcontroler folosind Arduino, o platformă de dezvoltare familiară pentru programarea dispozitivelor IoT.

3.2. Senzorul BME280

Senzorul BME280 este un senzor de mediu produs de Bosch care măsoară temperatura, umiditatea, presiunea atmosferică și altitudinea. Valoarea temperaturii ambientale poate fi în grade Celsius ($^{\circ}\text{C}$), umiditatea relativă a aerului este exprimată în procente, iar presiunea atmosferică în unități precum hectoPascal (hPa) sau milibari (mb). Printre beneficiile acestui senzor se numără timpul de răspuns foarte scăzut, și anume 1 secundă și o precizie de măsurare foarte bună pentru fiecare parametru în parte. Toate acestea, la care se adaugă și o greutate foarte mică (1g) și un preț relativ mic fac ca acest senzor să reprezinte o soluție excelentă pentru aplicațiile de monitorizare a factorilor de mediu.

Senzorul permite comunicare prin două protocoale: I2C și SPI. Dacă nu este necesară conectarea mai mult senzori, atunci cea mai simplă metodă este folosirea protocolului I2C. Interfața I2C (Inter-Integrated Circuit) este „un protocol de magistrală serială pe distanță scurtă” [8] utilizat pentru a permite comunicarea între dispozitivele electronice. Aceasta este o metodă simplă și eficientă de transmitere a datelor pe distanță scurtă între un dispozitiv master și dispozitive slave conectate pe același bus. Senzorul BME280 suportă două adrese pentru I2C: 0x76 și 0x77.

În figura 3.2. se poate observa că senzorul dispune de patru pini care au următoarele roluri:

- VCC: Alimentare (+) pentru senzor (3.3V);
- GND: Conexiune la masă (-);
- SDA: Linia de date serială pentru comunicația I2C;
- SCL: Linia de ceas serial pentru comunicația I2C.

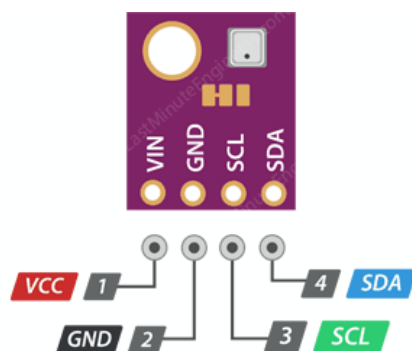


Fig. 3.2: Pinii senzorului BME280 [9]

2. TEHNOLOGII FOLOSITE

4.1. MQTT

MQTT (Message Queuing Telemetry Transport) este un protocol de comunicare adoptat pe scară largă și ușor de utilizat, care a fost conceput inițial pentru monitorizarea și controlul aplicațiilor în rețele de dispozitive compacte și inteligente. Acesta a devenit extrem de popular în domeniul Internet of Things (IoT), datorită caracteristicilor sale eficiente și scalabile.

Protocolul MQTT se bazează pe un model publish-subscribe, unde dispozitivele sunt împărțite în două roluri principale: publicatori (publishers) și abonați (subscribers). În Figura 4.1. este prezentat modul în care aceștia comunică între ei. Publicatorii sunt responsabili pentru trimiterea mesajelor către un broker MQTT central, iar abonații se conectează la anumite subiecte de interes și astfel „primesc fiecare mesaj care este publicat la acele subiecte”[10].

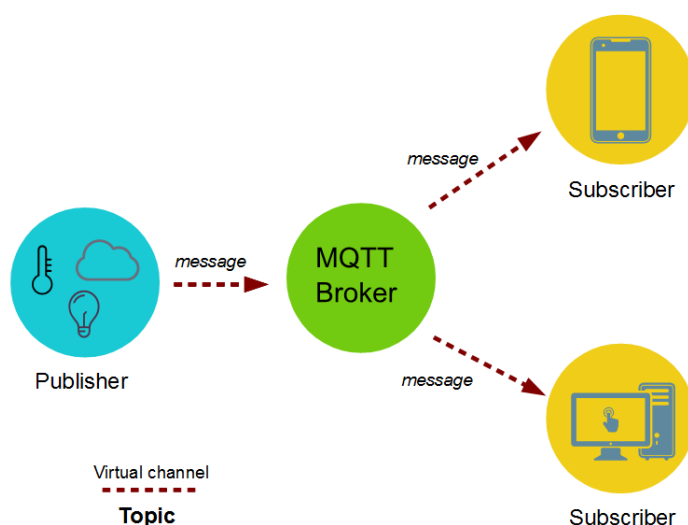


Fig. 4.1: Arhitectura MQTT [11]

Unul dintre avantajele semnificative ale MQTT este eficiența sa în utilizarea lății de bandă și în gestionarea resurselor. Protocolul a fost dezvoltat pentru a minimiza traficul de rețea prin utilizarea unui format compact de mesaje și o strategie de transmisie bazată pe calitatea de serviciu (QoS) care asigură că acestea ajung la destinație într-un mod sigur și în ordinea corectă. Mai mult decât atât, MQTT oferă flexibilitate atunci când vine vorba de securitate și autentificare și suportă diverse mecanisme de autentificare și criptare. Acest lucru permite implementarea unor niveluri ridicate de securitate în aplicațiile IoT.

4.2. MOSQUITTO

Mosquitto este un broker MQTT open-source, care oferă un serviciu de mesagerie și comunicare între dispozitivele care utilizează protocolul MQTT. Este dezvoltat și menținut de Eclipse Foundation și permite conectarea senzorilor și a dispozitivelor „într-o arhitectură de publicare/abonare cu mai multă flexibilitate decât în modelul de comunicare cerere/răspuns al arhitecturilor tradiționale client/server”[12].

Mosquitto acționează ca un intermediar între publicatorii (publishers) și abonații (subscribers) din rețeaua MQTT. Una dintre caracteristicile cheie ale acestuia este scalabilitatea sa. Acesta este capabil să gestioneze un număr mare de conexiuni simultane și să administreze eficient publicațiile și abonările în rețea.

Mosquitto este disponibil pe mai multe platforme, inclusiv Linux, Windows, macOS și diverse sisteme de operare încorporate, fiind utilizat într-o varietate de aplicații, de la proiecte personale până la soluții comerciale și industriale. Acesta poate fi configurat, de asemenea, într-un mediu Docker împreună cu toate dependențele sale. În acest mod procesul de instalare și configurare devine mult mai rapid și mai simplu, iar brokerul Mosquitto poate rula oriunde fără a fi nevoie de instalarea manuală a tuturor dependențelor.

4.3. PlatformIO

PlatformIO este un mediu de dezvoltare integrat (IDE) open-source pentru dezvoltarea și gestionarea sistemelor încorporate (embedded). Este conceput pentru a simplifica procesul de dezvoltare a aplicațiilor pentru diferite platforme și plăci de dezvoltare, cum ar fi Arduino, ESP8266, ESP32, Raspberry Pi și multe altele.

PlatformIO oferă un set de instrumente și funcționalități care facilitează programarea, depanarea și gestionarea bibliotecilor și dependențelor într-un mod eficient. Mediu de dezvoltare are o interfață prietenoasă, care oferă suport pentru editare de cod, compilare, încărcare a codului pe plăci de dezvoltare, monitorizare a ieșirii dispozitivelor, depanare și multe altele. Prin intermediul PlatformIO, dezvoltatorii pot lucra cu mai multe platforme hardware și pot utiliza diverse limbaje de programare, inclusiv C++, Python. PlatformIO facilitează gestionarea proiectelor și bibliotecilor, permițând dezvoltatorilor să creeze aplicații încorporate complexe și eficiente.

Unul dintre principalele avantaje ale PlatformIO este abordarea sa unificată și interoperabilă. Aceasta înseamnă că dezvoltatorii pot utiliza aceeași infrastructură și flux de lucru pentru diferite platforme și proiecte, ceea ce face procesul de dezvoltare mai consistent și mai ușor de gestionat.

PlatformIO are o infrastructură integrată care permite dezvoltatorilor să adauge și să gestioneze biblioteci în proiectele lor. Pentru acest proiect au fost necesare următoarele biblioteci: PubSubClient, Adafruit BME280, ESP AsyncWebServer și ArduinoJson.

Biblioteca **PubSubClient** este adesea utilizată în dezvoltarea de aplicații IoT (Internet of Things) și este compatibilă cu diferite platforme hardware și mediul de dezvoltare PlatformIO. Deoarece este scrisă în C++, aceasta poate fi integrată ușor în proiecte existente și oferă funcționalități MQTT (Message Queuing Telemetry Transport) pentru

dispozitivele încorporate. De asemenea, oferă funcționalități pentru publicarea mesajelor către un topic specific și pentru abonarea la topic-uri pentru a primi mesaje de la brokerul MQTT. Aceasta permite dispozitivelor să comunice și să schimbe informații într-un mod eficient.

Biblioteca **Adafruit BME280** este o bibliotecă care permite comunicarea cu senzorul de temperatură, umiditate și presiune BME280 produs de Adafruit. Acesta oferă un set de funcționalități necesare pentru configurarea senzorului BME280 și pentru citirea valorilor sale.

Biblioteca **ESP AsyncWebServer** este o bibliotecă pentru platforma ESP8266 și ESP32 care permite dezvoltatorilor crearea de servere web asincrone care pot gestiona simultan multiple cereri HTTP. Aceasta înseamnă că serverul poate răspunde la mai multe cereri în același timp fără a bloca execuția programului principal.

Biblioteca **ArduinoJson** permite manipularea și gestionarea datelor JSON pe platforma Arduino și PlatformIO prin intermediul funcționalităților sale puternice de creare, interpretare, serializare și deserializare a structurilor de date JSON.

4.4. NestJS

NestJS este un framework de dezvoltare a aplicațiilor web și server-side, construit pe baza platformei Node.js. Acesta a fost creat cu scopul de a oferi dezvoltatorilor un cadru scalabil, modular și ușor de înțeles pentru construirea aplicațiilor backend, dar și o performanță ridicată.

NestJS oferă suport extins pentru TypeScript, un limbaj de programare bazat pe JavaScript, care adaugă tipizare statică și alte caracteristici avansate de programare. Acest lucru ajută la crearea de cod mai robust și mai ușor de gestionat.

Framework-ul pune un accent deosebit pe utilizarea decoratorilor și a metadatelor pentru a defini structura aplicației și pentru a gestiona dependențele. Acest lucru facilitează injectarea de dependențe și construirea unei arhitecturi flexibile și modularizate.

De asemenea, oferă suport pentru diverse module și biblioteci, care pot fi utilizate pentru a extinde funcționalitatea aplicației și care acoperă o gamă largă de funcționalități, precum baze de date, autentificare, validare, comunicare cu servicii externe și multe altele.

Un alt aspect important al NestJS este gestionarea cererilor HTTP prin intermediul controlerelor. Acestea permit definirea rutelor API și gestionarea logicii de business asociate cererilor HTTP. NestJS oferă, de asemenea, suport pentru crearea de middlewares, filtre și interceptori, care pot fi utilizați pentru a personaliza fluxul de cereri și răspunsuri în aplicație.

În NestJS controllerele, modulele și serviciile sunt componente esențiale în structura unei aplicații. Controllerele sunt responsabile pentru gestionarea rutelor și a logicii de control asociate. Acestea primesc cereri HTTP de la client și „returnează datele către client”[13].

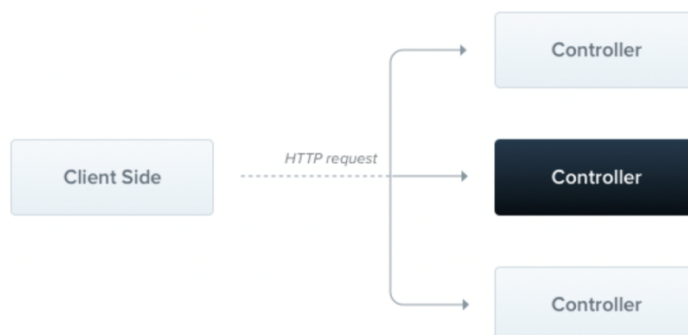


Fig.4.4.1: Exemplu de funcționare a controllerelor în Nest [13]

Modulele reprezintă componente care ajută la organizarea codului în cadrul aplicației Nest.js. Acestea sunt responsabile de încapsularea funcționalității și de gestionarea dependențelor între diversele componente ale aplicației. Modulele sunt „locul în care se conectează serviciile, controlerele”[13] și alte componente necesare pentru implementarea unei anumite funcționalități. Acestea pot fi văzute ca unități independente și reutilizabile ale aplicației. În cadrul aplicației acestea sunt anotate cu decoratorul `@Module`.

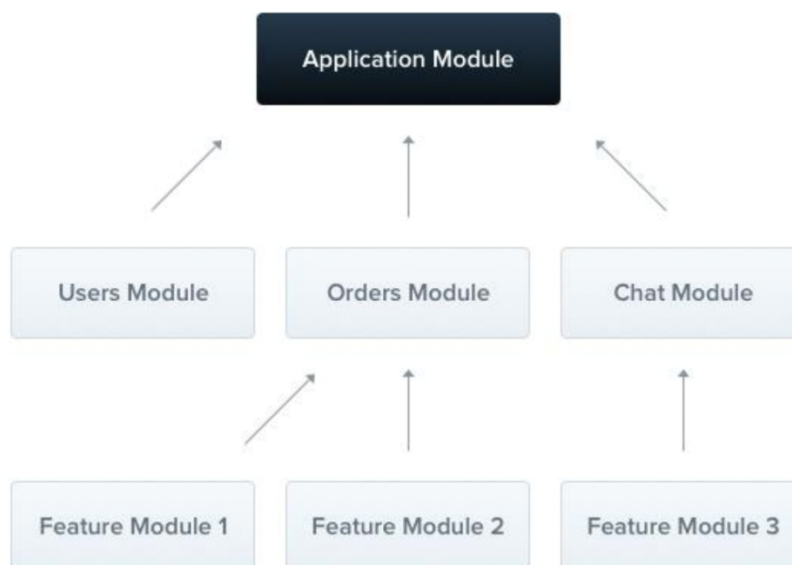


Fig.4.4.2: Exemplu de funcționare a modulelor în Nest [13]

Serviciile sunt utilizate pentru a implementa logica aplicației. Acestea furnizează funcționalități și metode specifice care sunt utilizate de către controlere sau alte servicii. Serviciile pot fi injectate în alte componente și pot fi folosite pentru a accesa baze de date, a efectua operații CRUD, a comunica cu servicii externe etc. În NestJS sunt evidențiate cu ajutorul decoratorului `@Injectable()`.

Micro serviciu MQTT [14]

Un micro serviciu MQTT în cadrul framework-ului Nest este o componentă care gestionează comunicarea cu un broker MQTT. Acesta oferă funcționalitatea de a se conecta la un broker MQTT, de a publica și a abona la topic-uri MQTT, precum și de a gestiona mesajele primite și trimise. Serviciul poate fi configurat cu parametri precum adresa și portul brokerului MQTT, credențialele de autentificare și alte opțiuni specifice. Prin intermediul microserviciului MQTT, aplicația Nest poate comunica și interacționa cu dispozitive IoT și alte aplicații care folosesc protocolul MQTT.

Mongoose [15]

Mongoose este o bibliotecă pentru comunicarea cu baza de date MongoDB. Aceasta oferă un set de funcționalități puternice pentru a defini și manipula modelele de date, a realiza operații CRUD (Create, Read, Update, Delete) și a implementa logica specifică accesului la date.

Principalele componente sunt schemele și modelele. O schemă definește structura și regulile de validare pentru un model de date MongoDB. Aceasta poate fi considerată ca un șablon pentru obiectele salvate în baza de date. Schemele definesc tipurile de date, proprietățile și constrângerile pentru câmpurile din documentele MongoDB. Modelul este o instanță a unei scheme și reprezintă o colecție în baza de date MongoDB. De asemenea, modelele oferă metode și funcții utile pentru manipularea datelor, precum crearea, găsirea și actualizarea acestora.

JWT Token

Nest oferă un mecanism sigur și scalabil de autentificare și autorizare în aplicații prin utilizarea token-ului JWT. Prin generarea și validarea token-urilor, se poate asigura accesul la resursele protejate doar utilizatorilor autentificați și autorizați, oferind în același timp un nivel înalt de securitate și performanță.

4.5. REACT

React este o bibliotecă JavaScript open-source utilizată pentru construirea interfețelor pentru utilizatori în aplicații web. Aceasta a fost dezvoltată de către Facebook și este folosită pe scară largă în industria dezvoltării de software.

React „funcționează bazat pe stări și componente”[16], permițând dezvoltatorilor să creeze componente reutilizabile și să gestioneze eficient starea aplicației. Este, de asemenea, cunoscută pentru performanța ridicată și abilitatea de a actualiza dinamic interfața utilizatorului în funcție de schimbările care au loc în starea aplicației.

React TypeScript este o abordare de dezvoltare care utilizează biblioteca React împreună cu TypeScript, care aduce avantaje precum tipizarea statică, verificarea tipurilor la compilare și sugestii automate în dezvoltarea aplicațiilor React. Prin adăugarea tipizării

statice, dezvoltatorii pot detecta și evita erorile de tip în timpul procesului de dezvoltare, obținând un cod mai robust și mai ușor de întreținut.

Unul dintre principalele avantaje ale utilizării React este compoziția. Compoziția în React este o abordare de construire a interfețelor utilizator flexibile și modulare prin împărțirea componentelor în componente mai mici și independente, care pot fi apoi compuse împreună pentru a crea interfețe complexe. Similar cu modul în care piesele puzzle-ului se potrivesc împreună pentru a crea o imagine completă, componentele din React sunt compuse împreună pentru a forma interfața utilizator finală (Fig.4.5.1). Componentele copii reprezintă piesele individuale ale puzzle-ului, fiecare având propriul lor scop și funcționalitate. Componentele părinte sunt cele care coordonează și assemblează aceste piese, precum stabilirea poziției lor și gestionarea relațiilor dintre ele. Astfel, compoziția în React ne permite să adăugăm și să rearanjăm componentele pentru a obține interfețe flexibile și modulare, la fel cum putem reorganiza piesele puzzle-ului pentru a obține diferite variante ale imaginii finale.



Fig. 4.5.1: *React composition* [17]

Acest proces se realizează prin utilizarea componentelor funcționale și a conceptelor precum componentele copii și componente părinte, evitând astfel ierarhii adânci de componente și moștenirea proprietăților (prop drilling). Prin utilizarea compoziției în loc de moștenire, React promovează reutilizarea și modularitatea. În loc să se extindă o clasă de bază, funcționalități și comportamente pot fi adăugate prin compunerea componentelor existente, ceea ce permite o flexibilitate mai mare în structura și organizarea codului.

React se bazează pe principiul responsabilității unice (Single Responsibility Principle). Principiul a fost formulat de Robert C. Martin și este unul dintre principiile S.O.L.I.D în dezvoltarea software-ului. Acesta afirmă că „o clasă sau un modul ar trebui să aibă o singură responsabilitate și să aibă un singur motiv pentru a fi modificat”[18]. Ca urmare, se pot crea componente reutilizabile care permit împărțirea aplicației în părți mici și ușor de gestionat, având fiecare o singură sarcină specificată.

React Hook Form [19]

React Hook Form este o bibliotecă populară pentru gestionarea formulelor în aplicațiile React. Aceasta oferă hook-uri personalizate care facilitează gestionarea stării unui formular, validarea datelor și trimiterea acestora.

O caracteristică importantă a React Hook Form este sistemul său de validare simplificat. Acesta permite definirea de reguli de validare pentru câmpurile formularului, cum ar fi valori obligatorii, formate specifice sau reguli personalizate. Validarea se face automat în momentul trimiterii formularului sau în timpul completării acestuia.

useForm este un hook din biblioteca React Hook Form care permite inițializarea și gestionarea unui formular. Prin utilizarea acestui hook, se poate obține acces la diverse funcționalități utile pentru manipularea și validarea datelor introduse în formular. Acesta returnează mai multe valori utile, cum ar fi metodele **register**, **handleSubmit**, **getValues**, **watch**, **reset** și altele. În continuare se vor prezenta câteva caracteristici cheie ale fiecărei metode.

register este o metodă esențială, care ajută la înregistrarea câmpurilor formularului. Aceasta primește ca parametru numele câmpului și opțiuni suplimentare, cum ar fi regulile de validare.

handleSubmit este o metodă care trebuie apelată în momentul submit-ului formularului și gestionează logica asociată. Pentru această metodă este necesară definirea unei funcții de callback pentru a executa anumite acțiuni la finalizarea completării, cum ar fi trimiterea datelor la server.

Metoda **getValues** permite obținerea valorilor curente ale tuturor câmpurilor dintr-un formular. Prin utilizarea acestei metode, se pot accesa și utiliza valorile introduse de utilizator în câmpurile formularului.

Metoda **watch** permite urmărirea și obținerea actualizărilor pentru câmpurile specifice dintr-un formular. Prin utilizarea acestei metode, este posibilă reacționarea la modificările valorilor câmpurilor și efectuarea acțiunilor corespunzătoare.

Metoda **reset** permite resetarea valorilor și starea unui formular la starea inițială. Prin utilizarea acestei metode, toate câmpurile vor reveni la valorile inițiale, mesajele de eroare vor fi șterse și validarea formularului va fi resetată.

SWR [20]

Biblioteca SWR (Stale-While-Revalidate) este o bibliotecă React pentru gestionarea cache-ului și a cererilor de date în aplicațiile web. Scopul principal al SWR este de a facilita obținerea și gestionarea datelor de la server, oferind o experiență de utilizare fluidă și reactivă.

SWR folosește o strategie inteligentă de cache pentru a stoca local datele obținute de la server. Atunci când o cerere este efectuată, SWR verifică întâi cache-ul local și returnează datele existente, ceea ce duce la o încărcare mai rapidă a paginii. În același timp, SWR trimite o cerere către server pentru a actualiza datele cache-ului. Odată ce datele cache-ului sunt returnate, SWR inițiază o revalidare automată în fundal. Asta înseamnă că,

În timp ce datele cache-uite sunt afișate utilizatorului, SWR verifică și actualizează datele de pe server. Astfel, aplicația rămâne întotdeauna actualizată cu cele mai recente informații disponibile.

Un alt aspect important este suportul integrat pentru gestionarea erorilor. Aceasta include reîncercarea cererilor eșuate, gestionarea codurilor de eroare și posibilitatea de a trata erorile în mod personalizat. Funcția **mutate** din SWR este folosită pentru a actualiza sau a muta datele în cache și pentru a declanșa revalidarea ulterioară a datelor prin efectuarea unui nou apel către server. Astfel, se obțin actualizări în timp real ale datelor, eliminând nevoia de a face apeluri repetitive către server pentru a obține datele actualizate.

Utilizarea acestei biblioteci oferă un plus de performanță aplicației și îmbunătățește experiența utilizatorilor.

React Use Wizard

React Use Wizard este o bibliotecă pentru React care oferă funcționalități și metode pentru implementarea unui flux de lucru pas cu pas într-o aplicație. Aceasta simplifică gestionarea și navigarea între etape, permițând dezvoltatorilor să creeze aplicații mai structurate și mai ușor de utilizat.

Prin utilizarea acesteia, dezvoltatorii pot crea interfețe utilizator intuitive și structurate, ghidând utilizatorii prin diferitele etape ale unei acțiuni sau ale unui proces. Biblioteca facilitează avansarea sau retrogradarea între etape, oferind funcții și metode pentru a controla fluxul procesului.

Componenta **Wizard** este compusă din mai multe etape, fiecare reprezentând o secțiune distinctă a procesului. Utilizatorii pot naviga între etape folosind butoane de navigare sau indicatori vizuali care indică etapa curentă și progresul general în cadrul wizard-ului. Fiecare etapă poate avea conținut specific, cum ar fi formulare, întrebări sau alte elemente interactive. Utilizatorii pot completa informațiile necesare în fiecare etapă înainte de a avansa la următoarea etapă.

Hook-ul **useWizard** este o funcție personalizată care facilitează gestionarea stării și navigarea între componente. Cu ajutorul acestuia se pot obține informații despre pasul curent, numărul total de pași, dar face posibilă și trecerea la pasul următor sau revenirea la cel precedent.

Material UI

Material-UI este o bibliotecă de componente React care oferă un set extins de componente vizuale reutilizabile, stilizate și personalizabile, precum butoane, căsuțe de text, iconițe, meniuri și multe altele. Material-UI facilitează dezvoltarea interfețelor utilizator intuitive și atractive, cu un aspect modern și coerent.

Pentru acest proiect, cele mai utilizate componente ale bibliotecii Material UI au fost iconițele care oferă o varietate de simboluri pentru a îmbunătăți interfața utilizatorului și pentru a transmite informații vizuale. În figura 4.5.2 se poate observa că platforma deține o

pagină dedicată iconițelor, care include o listă completă a tuturor iconițelor disponibile, împreună cu numele lor.

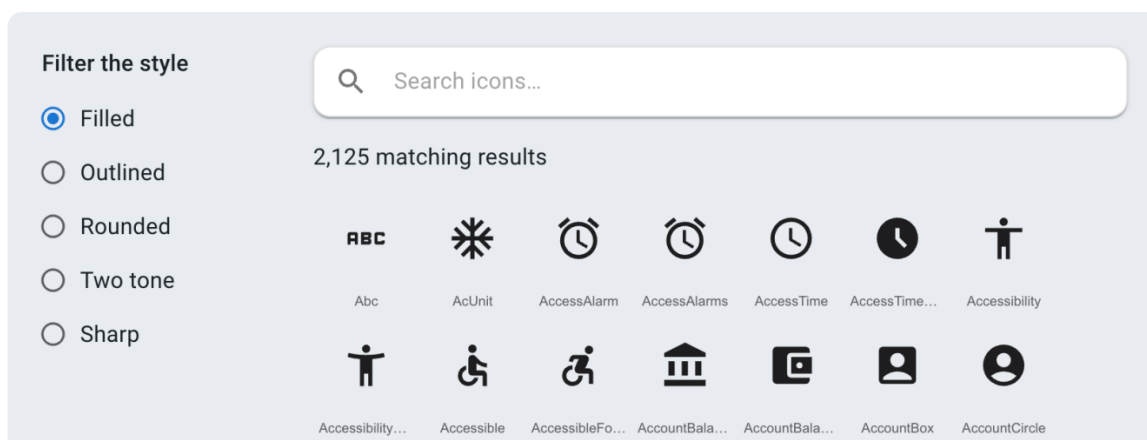


Fig.4.5.2: Modul de căutare al iconițelor [21]

Classnames

Biblioteca **classnames** este o bibliotecă populară în mediul React deoarece permite gestionarea și generarea dinamică a claselor CSS în componente. Principala funcție a bibliotecii este de a combina și genera claselor CSS pe baza unui set de condiții sau proprietăți. Aceasta facilitează gestionarea claselor dinamice în funcție de starea sau comportamentul componentelor React.

classnames permite concatenarea și combinarea claselor CSS pentru a crea clase complexe și flexibile. Clasele sunt definite în funcție de anumite condiții care pot include valori booleene, obiecte cu perechi cheie-valoare sau expresii condiționale. Clasele corespunzătoare vor fi generate doar pentru condițiile care sunt evaluate ca fiind adevărate.

Cel mai important avantaj al acesteia este că ajută la evitarea duplicării codului, la crearea unor clase CSS complexe și permite adaptarea stilurilor în funcție de starea sau interacțiunea componentelor.

4.6. VITE

Vite este un mediu de dezvoltare rapid (bundling tool) pentru aplicații web. A fost creat pentru a îmbunătăți experiența de dezvoltare și performanța în dezvoltarea aplicațiilor web moderne. Vite utilizează module ESM (ECMAScript Modules) pentru a permite importul direct al fișierelor JavaScript și a altor module, fără a fi necesar un proces de transformare sau de bundling extensiv. Acest lucru contribuie la o încărcare rapidă a aplicației în timpul dezvoltării și în producție. De asemenea, Vite vine cu un server de dezvoltare rapid care oferă reîncărcare rapidă la salvarea modificărilor. Aceasta înseamnă că schimbările efectuate în cod sunt reflectate instantaneu în browser, fără a fi necesar un proces de reconstruire sau de reîncărcare manuală a paginii.

Combinarea dintre React și Vite poate aduce numeroase beneficii în dezvoltarea unei aplicații deoarece Vite oferă un server de dezvoltare performant, cu funcționalități precum reîncărcare rapidă la salvarea modificărilor. Aceste caracteristici permit

dezvoltatorilor să se concentreze mai mult pe cod și să beneficieze de un flux de dezvoltare rapid și interactiv.

În plus, Vite funcționează bine cu React, oferind suport nativ pentru importul componentelor fără a fi necesară o configurație suplimentară. Astfel, dezvoltatorii pot crea rapid interfețe React, beneficiind în același timp de performanța și ușurința de utilizare a Vite.

4.7. BAZE DE DATE

În acest proiect s-au folosit două baze de date. Una dintre ele este InfluxDB care a fost folosită pentru salvarea valorilor primite de la senzori, iar cealaltă este MongoDB folosită pentru salvarea utilizatorilor și a datelor importante, cum ar fi adresa IP a senzorului și material de construcție al pereților dintr-o încăpăre.

InfluxDB

InfluxDB este o bază de date specializată în gestionarea datelor de tip serie temporală, adică date care sunt asociate cu un timp specific sau o secvență temporală. Aceasta oferă o soluție eficientă și scalabilă pentru stocarea și interogarea datelor în timp real, fiind folosită într-o varietate de aplicații precum monitorizarea sistemelor, telemetria dispozitivelor IoT, analiza datelor de senzori și multe altele.

Caracteristica distinctivă a InfluxDB este modelul său simplu de date, care se concentrează pe conceptul de "măsurători". O măsurătoare reprezintă un set de puncte de date asociate cu un timp specific și este formată dintr-un nume, o serie de etichete și un set de câmpuri. Etichetele permit indexarea și filtrarea datelor, iar câmpurile conțin valorile efective ale măsurătorilor. Această structură simplă facilitează interogarea și analiza datelor de tip serie temporală.

InfluxDB are o arhitectură distribuită și suportă scalabilitate orizontală, permițând manipularea unor volume mari de date. De asemenea, oferă funcționalități avansate precum retenționarea și comprimarea datelor, care permit păstrarea istoricului pe termen lung și optimizarea spațiului de stocare.

MongoDB

MongoDB este o bază de date NoSQL (non-relațională) care se bazează pe un model document-oriented pentru stocarea și gestionarea datelor. În loc să utilizeze tabele și relații ca în bazele de date relaționale tradiționale, MongoDB stochează datele în documente flexibile de tip JSON numite BSON (Binary JSON). Aceasta oferă o flexibilitate și scalabilitate mare în gestionarea datelor, deoarece nu necesită o structură rigidă sau o schemă predefinită.

MongoDB este proiectat să fie ușor de utilizat și să ofere performanță ridicată. Suportă un set bogat de funcționalități, inclusiv indexare, interogări avansate, agregare de date, replicare și fragmentare (sharding) pentru scalabilitate orizontală, gestionarea

tranzacțiilor și multe altele. Cu MongoDB, dezvoltatorii pot construi aplicații scalabile și flexibile, care se pot adapta ușor la cerințele în continuă schimbare. Baza de date oferă o abordare simplificată a gestionării datelor, eliminând nevoia de a defini scheme și relații stricte, permițând dezvoltatorilor să se concentreze mai mult pe logica aplicației și pe livrarea de funcționalități valoroase pentru utilizatori.

4.8. DOCKER

Docker este o platformă open-source care facilitează crearea, distribuirea și rularea aplicațiilor în containere. Un container Docker conține toate dependențele necesare pentru ca o aplicație să ruleze, inclusiv codul sursă, bibliotecile, sistemul de operare și orice alte pachete sau instrumente necesare. Cu ajutorul Docker, dezvoltatorii pot împacheta aplicațiile și toate dependențele lor în containere portabile, care pot fi executate pe orice sistem compatibil cu Docker, indiferent de mediul de dezvoltare sau de producție.

Docker utilizează tehnologia de virtualizare la nivelul sistemului de operare (cunoscută și sub denumirea de virtualizare la nivelul containerelor) pentru a asigura izolarea și portabilitatea aplicațiilor. Acest lucru înseamnă că fiecare container rulează într-un mediu izolat, dar partajează resursele sistemului de operare gazdă, ceea ce permite rularea mai multor containere pe același sistem fără a se suprapune sau a se interfera reciproc.

Prin utilizarea Docker, dezvoltatorii beneficiază de avantaje precum portabilitatea, reproducibilitatea și scalabilitatea. Aplicațiile împachetate în containere Docker pot fi ușor mutate între diferite medii de dezvoltare, testare și producție, asigurându-se că aplicația se comportă în același fel în orice context. De asemenea, Docker facilitează reproducerea exactă a unui mediu de dezvoltare sau de testare, astfel încât toți membrii echipei să lucreze într-un mediu consistent și identic.

Prin abstractizarea infrastructurii și pachetarea aplicațiilor și dependențelor lor în containere, Docker simplifică procesul de dezvoltare și implementare a aplicațiilor, permițând echipei să se concentreze mai mult pe codare și pe livrarea de valoare pentru utilizatori.

Pentru definirea și gestionarea mai multor containere într-un mod simplificat se poate utiliza **Docker Compose**. Cu ajutorul acestuia, se pot defini și configura rapid un set de containere care trebuie să ruleze împreună pentru a forma o aplicație complexă. Fișierul "docker-compose.yml" permite specificarea imaginilor Docker, parametrilor de configurare, dependențelor între containere și multe altele (Fig.4.8). Acesta oferă o modalitate simplă și consistentă de a gestiona întreaga infrastructură necesară pentru a rula aplicația dvs.

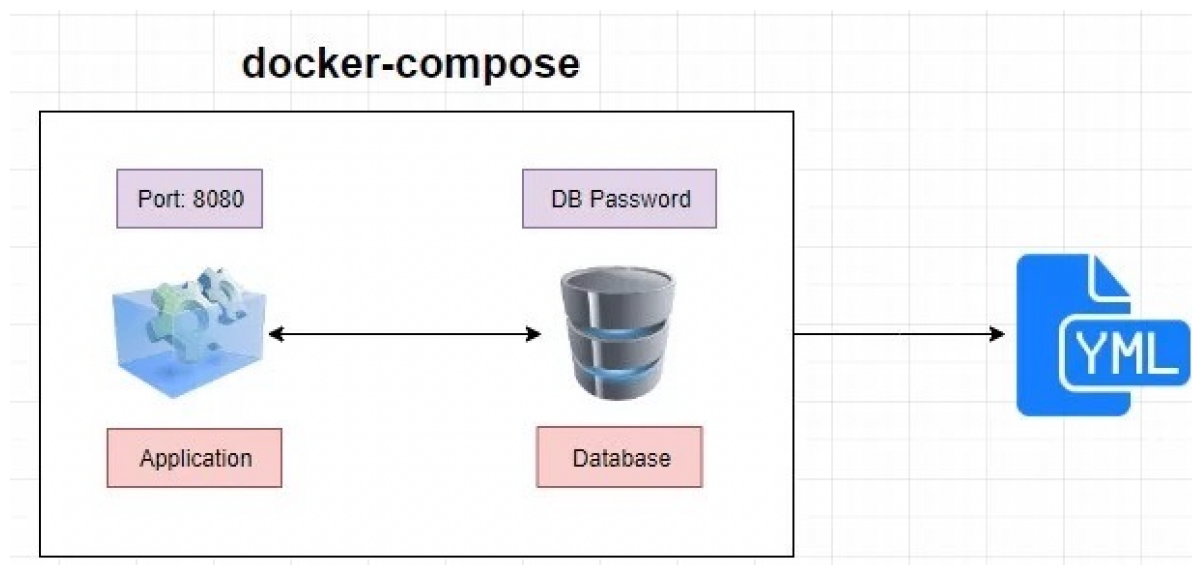


Fig. 4.8: Modul de funcționare al Docker Compose [22]

Un alt avantaj al Docker Compose este posibilitatea de a defini și gestiona variabile de mediu, volume și rețele personalizate pentru containere în funcție de nevoile aplicației.

4.9. DEVELOPMENT CONTAINER

Un development container este un container Docker special conceput pentru dezvoltarea unei aplicații. Acesta conține toate dependențele și instrumentele necesare pentru dezvoltarea și testarea aplicației într-un mediu izolat și portabil. Dezvoltatorii pot utiliza un development container pentru a crea un mediu de lucru consistent și reproducibil, indiferent de platforma pe care rulează.

Extensia "Dev Containers" pentru Visual Studio Code (VS Code) permite definirea și configurarea unui mediu de dezvoltare într-un container Docker folosind un set flexibil de fișiere de configurare. Acest lucru permite izolarea și reproducerea mediului de dezvoltare, asigurând consistența și portabilitatea proiectului. Prin intermediul extensiei "Dev Containers", se poate deschide un director local într-un container Docker, ceea ce înseamnă că toate instrumentele, dependențele și setările de dezvoltare sunt rulate într-un mediu controlat și consistent.

4.10. PRETTIER

Prettier este o unealtă de formatare a codului care ajută la menținerea unui stil de codare coerent și ușor de citit în proiectele de dezvoltare software. Rolul său principal este de a automatiza procesul de formatare a codului, astfel încât să nu mai fie nevoie de efort manual și de timp suplimentar pentru a ajusta indentarea, spațiile, punctuația și alte aspecte de formatare.

Prettier este capabil să formateze codul sursă într-un mod consistent și personalizat, indiferent de limbajul de programare utilizat. Este compatibil cu o varietate de

limbaje de programare populare, inclusiv JavaScript, TypeScript, HTML, CSS, JSON și multe altele. Cu ajutorul unor reguli de formatare predefinite sau personalizate, acesta se poate configura pentru a se potrivi cu stilul personal al dezvoltatorului proiectului sau cu normele de codare existente.



```
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

```
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

Fig. 4.10: Codul înainte și după formatarea cu Prettier

4.11. GIT

Git este un sistem de control al versiunilor distribuit, creat pentru a urmări și gestiona modificările într-un proiect de dezvoltare software. Rolul său principal este de a permite dezvoltatorilor să lucreze în echipă la un proiect, să își urmărească și să își gestioneze modificările și să faciliteze colaborarea și integrarea codului.

Prin utilizarea Git, fiecare membru al echipei de dezvoltare poate crea o copie locală a întregului repository, inclusiv a istoricului complet al modificărilor. Acest lucru permite fiecărui dezvoltator să lucreze independent în propriul său branch, să facă modificări, să creeze, să publice modificările și să revizuiască codul fără a afecta direct repository-ul principal. Git facilitează urmărirea și gestionarea modificărilor prin intermediul conceptelor de commit-uri, branch-uri, merge-uri și conflictelor de cod. Dezvoltatorii pot crea commit-uri pentru a înregistra schimbările făcute în cod, pot crea branch-uri pentru a lucra la funcționalități noi sau la rezolvarea de bug-uri în izolare, și pot realiza merge-uri pentru a aduce modificările dintr-un branch în altul.

3. PROIECTAREA SISTEMULUI

5.1. ARHITECTURA SISTEMULUI

Arhitectura unui sistem reprezintă o descriere detaliată a structurii, modul de organizare al componentelor și interacțiunilor dintre acestea. Aceasta definește modul în care elementele sistemului sunt proiectate, conectate și funcționează împreună pentru a îndeplini obiectivele sistemului.

În figura de mai jos (Fig.5.1) este prezentată arhitectura sistemului propus. Se poate observa că acesta include un mediu Docker în care rulează bazele de date și serverul MQTT. În afara acestuia se pot observa modulele ESP8266 care comunică cu brokerul MQTT, backend-ul, interfața aplicației și clienții. Serverul backend preia informațiile de la senzori cu ajutorul serverului MQTT și le salvează în baza de date InfluxDB. A doua bază de date, MongoDB, este utilizată în procesul de autentificare și autorizare a utilizatorilor și pentru stocarea notificărilor pentru fiecare utilizator în parte.

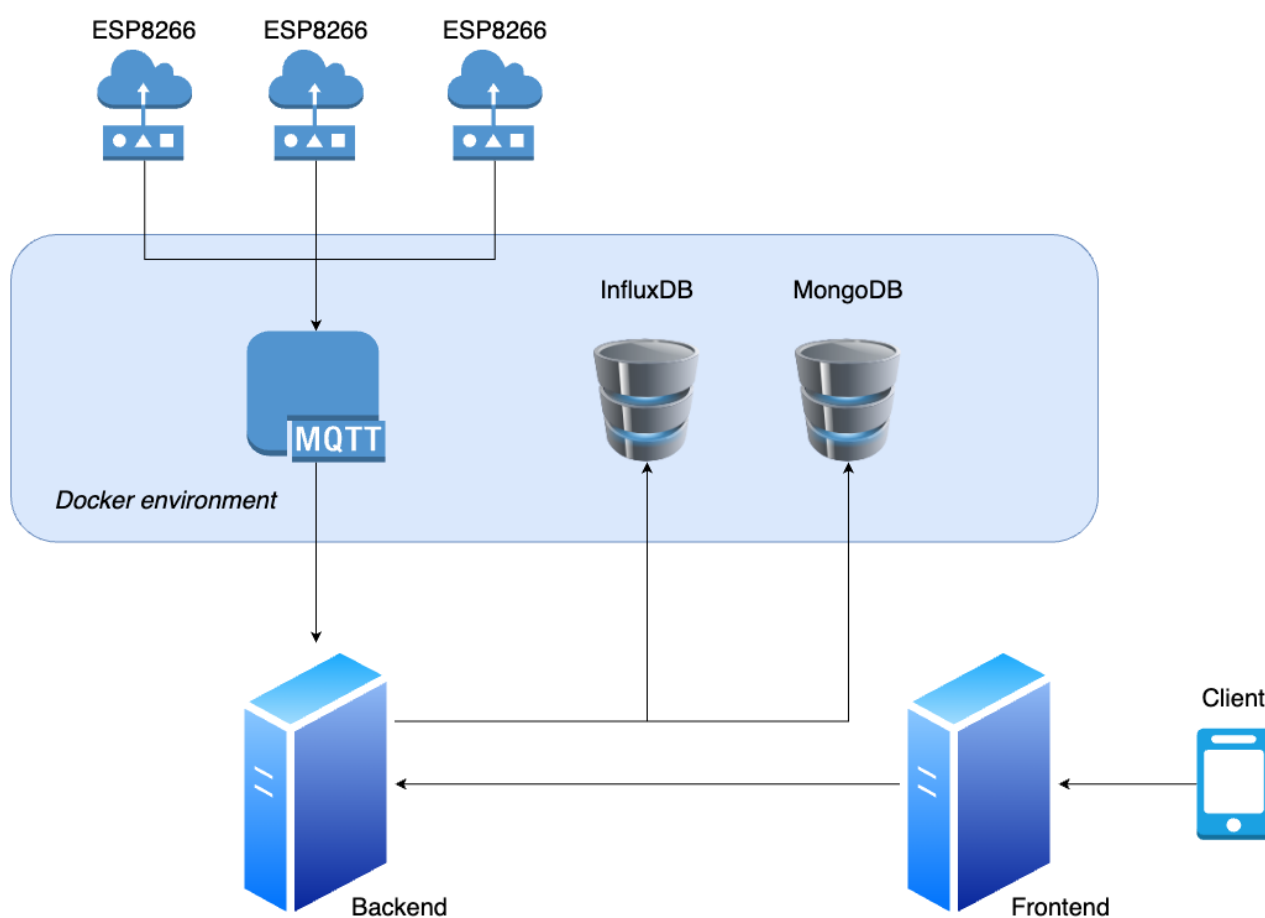


Fig. 5.1: Arhitectura sistemului

5.2. PROIECTAREA HARDWARE

Proiectarea hardware constă în conectarea modului ESP8266 cu senzorul BME280. Modulul ESP8266 are mai mulți pini GPIO (General Purpose Input/Output) care pot fi utilizați pentru conectarea senzorului BME280, dar în cazul de față au fost folosiți următorii pini:

- VCC (3.3V) - Vin (alimentare)
- GND - GND (masă)
- D1 (GPIO5) - SDA (linie de date I2C)
- D2 (GPIO4) - SCL (linie de ceas I2C)

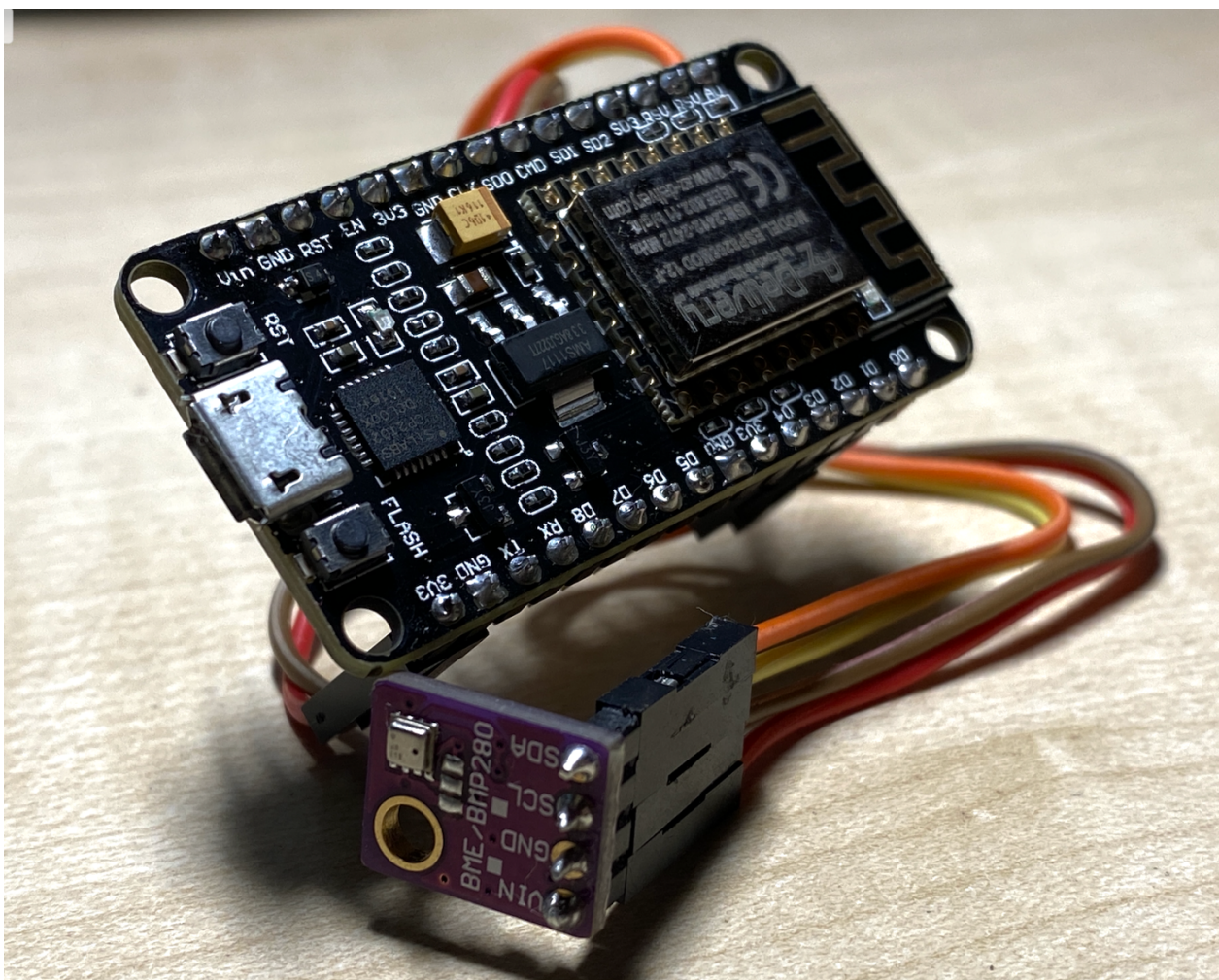


Fig.5.2.1: Conectarea modului ESP8266 și a senzorului BME280

Pentru construcția finală a modului ESP8266 și senzorul BME280 s-au folosit piese de LEGO care oferă o soluție practică și eficientă pentru a organiza și păstra aceste componente. Prin configurarea strategică a pieselor LEGO, se poate crea o structură personalizată care să găzduiască modulul și senzorul într-un spațiu compact și bine organizat. Astfel, se poate asigura o protecție împotriva deteriorării sau expunerii la factori externi. De asemenea, cu ajutorul pieselor se pot ghida cablurile într-un mod organizat pentru a evita încurcarea sau deteriorarea acestora.

În figura următoare (Fig.5.2.2) se poate observa rezultatul după crearea structurii din piese LEGO:

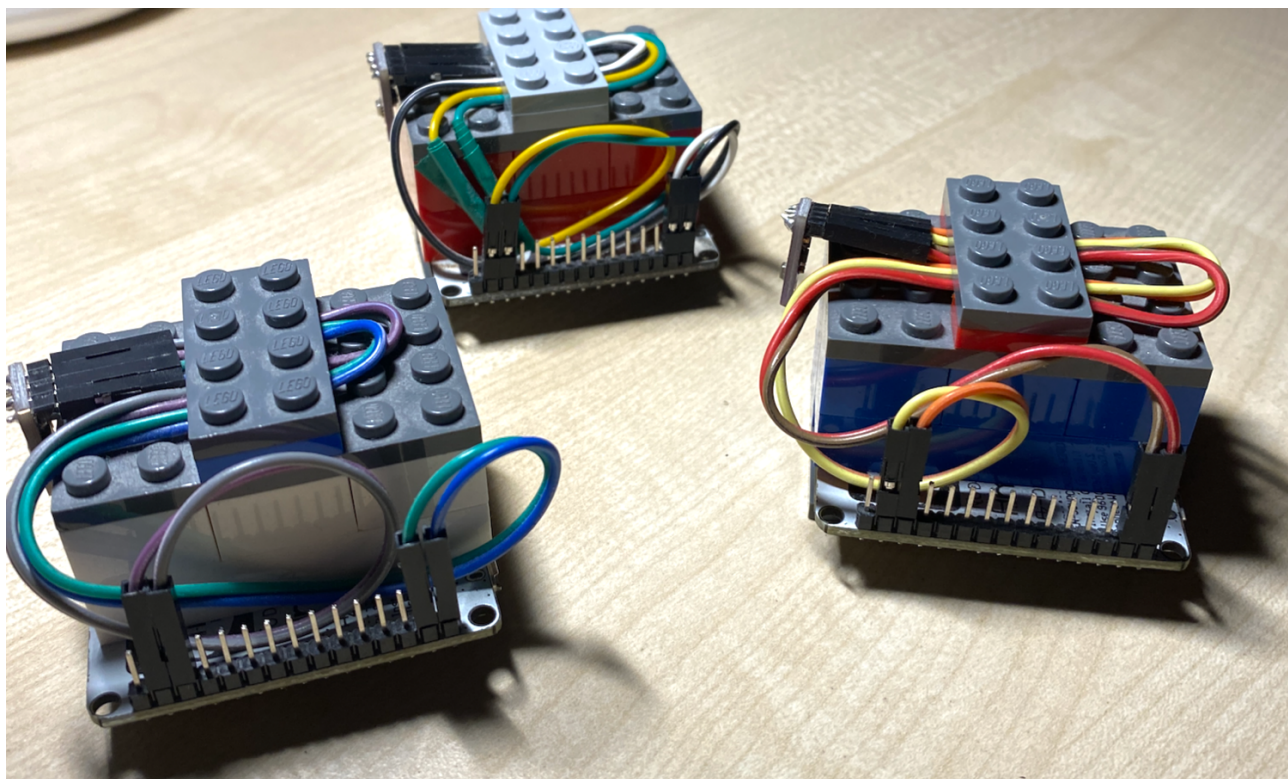


Fig.5.2.2: Structura finală a componentelor

5.3. PROIECTAREA SOFTWARE

Diagrama modulelor corespunzătoare fiecărui server este o reprezentare vizuală a componentelor implicate în arhitectura sistemului și interacțiunile dintre ele.

Figura 5.3.1 ilustrează diagrama modulelor pentru sistemul dezvoltat în această lucrare. Pentru microcipul ESP8266 se pot observa cele două servere: backend și frontend. ESP8266 frontend este o pagină HTML simplă care permite conectarea la WiFi prin selectarea rețelei dorite și introducerea parolei acesteia. Pe de altă parte, ESP8266 backend este un server asincron care pune la dispoziție un endpoint pentru conectarea la Internet și unul pentru procesul de on board al senzorului.

Sistemul dispune de un server de backend, tot cu o arhitectură modulară, în care avem patru module principale: senzor, MQTT, Mongo și Influx. Pentru acesta sunt prezentate în diagramă atât interacțiunile din interiorul serverului, cât și interacțiunile cu celelalte servere. Modulul NotificationJob conține un cron job care permite transmiterea notificărilor la un interval de timp specificat. Serverul de frontend interacționează cu modulele puse la dispoziție de backend și oferă utilizatorilor o interfață ușor de utilizat.

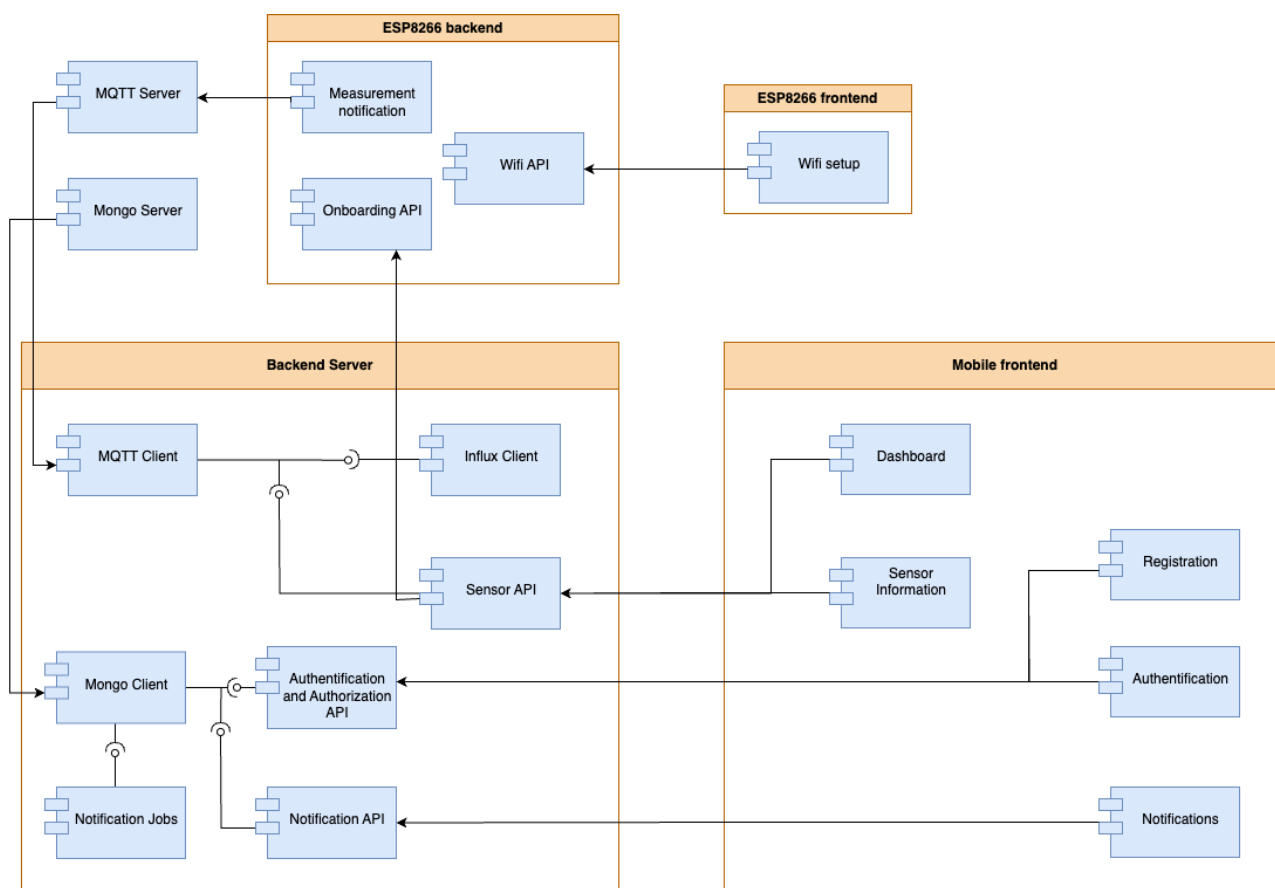


Fig.5.3.1: Diagrama modulelor

Diagrama cazurilor de utilizare este o reprezentare grafică a interacțiunilor între utilizatorii unui sistem și funcționalitățile acestuia. Ea oferă o perspectivă asupra acțiunilor pe care utilizatorii le pot efectua și a modului în care aceste acțiuni se încadrează în contextul general al sistemului. Actorii sunt reprezentați ca entități externe care interacționează cu sistemul, iar cazurile de utilizare sunt reprezentate ca acțiuni sau funcționalități pe care actorii le pot realiza. Aceste cazuri de utilizare sunt conectate prin relații pentru a indica fluxul de interacțiuni și dependențele între ele.

Relația "include" arată că cazul de utilizare inclus este o parte componentă a cazului de utilizare principal și este necesar pentru atingerea obiectivelor sistemului. Relația "extend" indică faptul că cazul de utilizare extins este o variantă a cazului de utilizare de bază și este activat doar în anumite condiții.

Mai jos (Fig.5.3.2) este prezentată diagrama cazurilor de utilizare pentru aplicația dezvoltată în acest proiect.

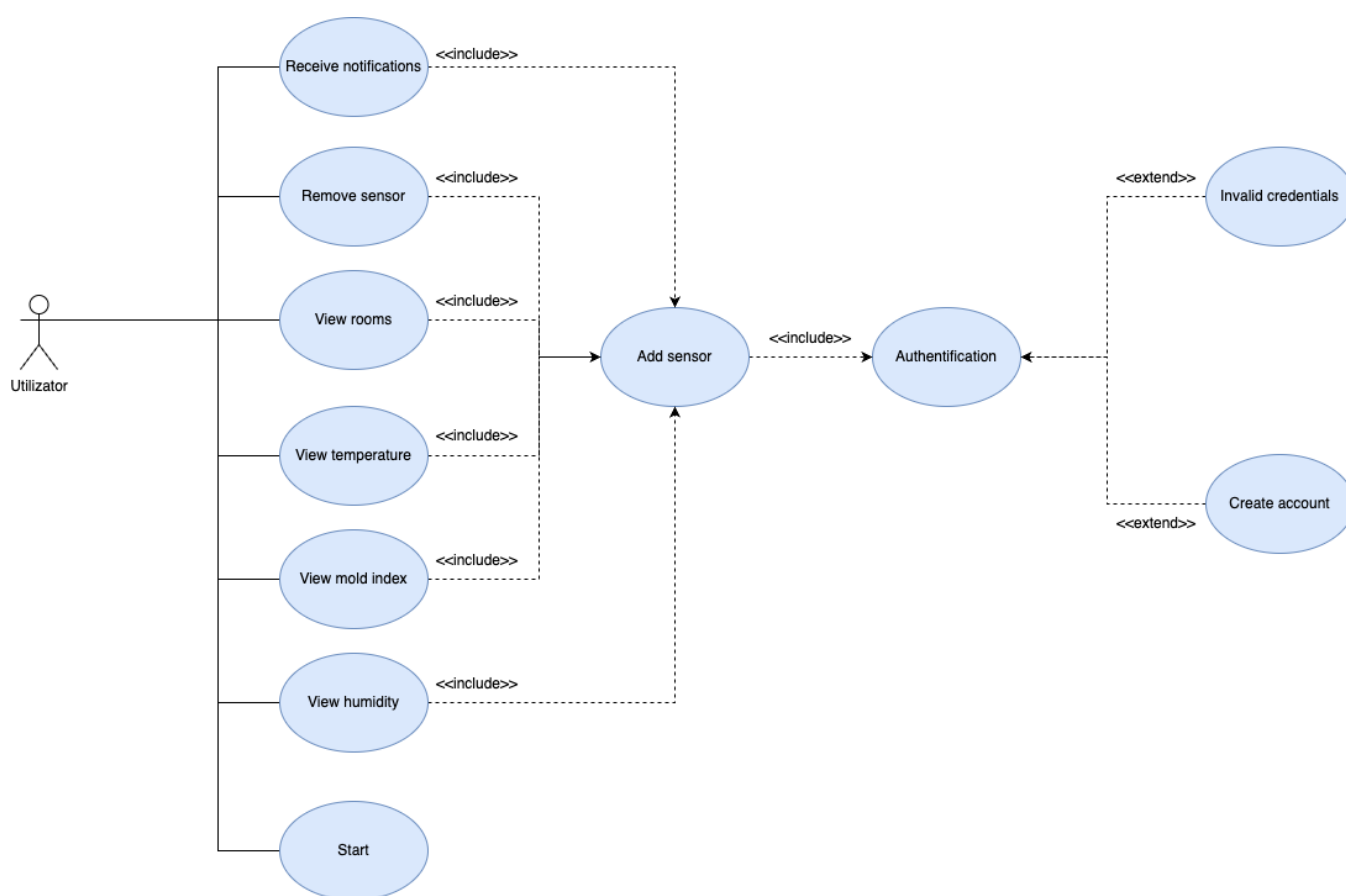


Fig.5.3.2: Diagrama de use case-uri

4. IMPLEMENTAREA FUNCȚIONALITĂȚILOR

6.1. CONFIGURARE CONTAINER DOCKER

Pentru configurarea infrastructurii Docker este necesară adăugarea unui fișier „**docker-compose.yml**” care permite definirea și gestionarea mai multor containere simultan, specificând setările, dependențele și interacțiunile dintre ele.

Pentru a ilustra modul de utilizare a fișierului, se prezintă modul de configurare a broker-ului MQTT în cadrul aplicației. Configurarea presupune adăugarea unui nou serviciu în fișier și specificarea setărilor acestuia, după cum se poate vedea în secvența de cod de mai jos.

```
services:
  mqtt:
    image: eclipse-mosquitto
    volumes:
      - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - ./mosquitto/log:/mosquitto/log/
      - mosquitto-data:/mosquitto/data/
    ports:
      - 1883:1883
      - 9001:9001
    command: >
      sh -c '
        if [ x${MOSQUITTO_PASSWORD} == x ]; then
          echo "Set the MOSQUITTO_PASSWORD environment variable in the .env file";
          exit 1;
        fi;
        echo "hta-sensor:${MOSQUITTO_PASSWORD}" > /mosquitto/config/password.txt
        mosquitto_passwd -U /mosquitto/config/password.txt
        /usr/sbin/mosquitto -c /mosquitto/config/mosquitto.conf
      '
    networks:
      - sensor-network
```

Secvența de cod 6.1: Configurarea broker-ului MQTT

În secvența de mai sus, este definit un serviciu numit **mqtt** care utilizează imaginea oficială Eclipse Mosquitto. Volumele permit partajarea și persistența datelor între container și gazdă. Aceasta înseamnă că orice modificare sau adăugare de date în containerul **mqtt** va fi reflectată și salvată pe gazdă, iar aceste date pot fi accesate și utilizate în mod persistent chiar și după repornirea sau reconstruirea containerului. În acest caz, au fost definite trei volume. Primul volum permite configurarea specifică a brokerului MQTT. Al doilea volum este pentru stocarea datelor generate de broker-ul MQTT, iar cel de-al treilea este pentru stocarea înregistrărilor și jurnalelor de activitate ale broker-ului. Acestea pot include informații despre conexiuni, abonamente, publicări, autentificare, erori etc.

Următorul pas este specificarea porturilor prin care brokerul MQTT va fi accesibil. Portul 1883 este portul standard pentru MQTT.

Câmpul **command** este utilizat pentru a specifica comanda care trebuie executată pentru un serviciu în momentul pornirii containerului. Pentru acest serviciu este creat un script care verifică dacă o parolă a fost setată sau nu prin variabila de mediu **MOSQUITTO_PASSWORD**. Dacă nu este setată, scriptul afișează un mesaj de eroare și se oprește cu codul de ieșire 1. În caz contrar, se creează un fișier utilizând numele de utilizator și parola citită și apoi pornește brokerul utilizând fișierul de configurare specificat.

Prin utilizarea câmpului **networks**, putem configura și gestiona mai eficient interacțiunea între containere într-un mediu Docker, asigurându-ne că acestea sunt conectate la rețea corespunzătoare și pot comunica în mod adecvat. Rețeaua în care sunt conectate toate serviciile definite în proiect este **sensor-network**.

6.2. CONFIGURARE DEVELOPMENT CONTAINER

Pentru a configura un development container în (VS Code) se creează un fișier JSON în directorul rădăcină al proiectului, fișier care va conține toate setările necesare pentru dezvoltarea într-un mediu controlat.

```
{
  "image": "mcr.microsoft.com/devcontainers/typescript-node:0-18",
  "runArgs": [
    "--network=sensor-network",
    "--privileged"
  ],
  "customizations": {
    "vscode": {
      "extensions": [
        "dbaeumer.vscode-eslint",
        "esbenp.prettier-vscode",
        "ms-vscode.vscode-typescript-next",
        "VisualStudioExptTeam.intellicode-api-usage-examples",
        "VisualStudioExptTeam.vscodetoolchain"
      ]
    }
  }
}
```

Secvența de cod 6.2: Configurare development container

În secvența de mai sus, se poate observa configurația pentru acest proiect. Cel mai important câmp este „**runArgs**” unde se poate specifica rețeaua la care trebuie să se conecteze containerul pentru a comunica cu alte containere care fac parte din aceeași rețea. Această rețea este cea specificată în configurația pentru Docker.

6.3. ÎNREGISTRAREA ÎN APLICAȚIE

Procesul de înregistrare este esențial pentru a permite utilizatorilor să beneficieze de funcționalitățile și serviciile aplicației. Acest proces de înregistrare implică completarea unui formular de înregistrare care presupune furnizarea unor informații de bază, cum ar fi adresa de email, numele și parola dorită. Pentru a înregistra și valida câmpurile din formular s-a folosit hook-ul **useForm**.

```
const {  
  register,  
  handleSubmit,  
  formState: { errors },  
  getValues,  
} = useForm<User>({ mode: "onChange" });
```

Secvența de cod 6.3.1: Inițializarea metodelor folosite

Secvența de cod de mai sus se folosește pentru inițializarea metodelor folosite. Setarea modului **onChange** este utilă atunci când se dorește un feedback instantaneu pentru utilizator cu privire la validitatea datelor introduse. Implicit, acesta este setat pe **onSubmit** ceea ce înseamnă că validarea câmpurilor se activează doar la momentul trimiterii formularului.

Înregistrarea unui câmp se realizează cu ajutorul funcției **register**. Aceasta primește ca parametri numele câmpului formularului, folosit pentru a identifica câmpul și a-l asocia cu valorile introduse și un obiect care conține diferite opțiuni pentru înregistrarea câmpului. Acest obiect este opțional. În acest exemplu, pentru validarea adresei de email, sunt utilizate două reguli:

1. **required**: Aceasta asigură că câmpul nu este gol și va afișa mesajul „Required field” în cazul în care utilizatorul încearcă să trimită formularul fără a completa acest câmp.
2. **pattern**: Aceasta utilizează o expresie regulată pentru a verifica dacă adresa de email respectă formatul specificat. Expresia verifică existența unui text înainte și după caracterul **@** urmat de un domeniu valid. În cazul în care valoarea introdusă nu respectă acest format, se va afișa mesajul „Invalid email address”.

```
<TextField  
  required  
  id="email"  
  label="Email"  
  error={!errors.email}  
  {...register("email", {  
    required: "Required field",  
    pattern: {  
      value: /^[a-z0-9._%+]+@[a-z0-9.-]+\.[a-z]{2,4}$/i,  
      message: "Invalid email address",  
    },  
  })}  
>
```

Secvența de cod 6.3.2: Înregistrare și validare câmp

Trimiterea formularului se realizează prin intermediul funcției **handleSubmit**. Aceasta primește o funcție de callback care este apelată când formularul este trimis cu succes. În interiorul funcției definite, se realizează apelul către endpoint-ul de tip POST al serverului și se tratează răspunsul.

Serverul are definit un controller care gestionează rutele HTTP și logica asociată acestora și un serviciu care este responsabil pentru validarea datelor și salvarea în baza de date.

În secvența de cod de mai jos, serviciul salvează noul utilizator în baza de date. Deoarece validarea se face înaintea trimiterii formularului, singura acțiune necesară este salvarea.

```
async create(createUserDto: UserDTO): Promise<User> {  
  const createdUser = new this userModel(createUserDto);  
  return createdUser.save();  
}
```

Secvența de cod 6.3.3: Metodă pentru crearea unui utilizator

În constructorul controllerului, se injectează serviciul creat. Această acțiune permite controllerului să utilizeze metodele și funcționalitățile furnizate de serviciu. Următoare secvență de cod ilustrează modul în care a fost adăugată o dependență a serviciului în constructorul controllerului și cum este apelată din controller metoda creată în serviciu.

```
@Controller('users')  
export class UserController {  
  constructor(private readonly userService: UserService) {}  
  
  @Post()  
  async create(@Body() createdDto: UserDTO) {  
    await this.userService.create(createdDto);  
  }  
}
```

Secvența de cod 6.3.4: Endpoint de tip POST pentru adăugarea unui utilizator

6.4. ESP8266 SERVER

Modulul poate funcționa în două moduri principale: modul STA (Station) și modul AP (Access Point). Pentru început acesta va funcționa în modul AP, devenind astfel un punct de acces WiFi la care alte dispozitive se pot conecta.

În secvența de mai jos, sunt definite numele rețelei WiFi (SSID), parola rețelei și adresă IP dorită pentru punctul de acces WiFi.

```
const char *ssid = "ESP8266-Access-Point";  
const char *password = "123456789";  
IPAddress apIP(192, 168, 1, 1);
```

Secvența de cod 6.4.1: Parametrii pentru ESP8266 în modul AP

După definirea parametrilor se poate seta adresa IP utilizând funcția **WiFi.softAPConfig()**. Aceasta primește ca parametri adresa IP definită anterior și masca de subrețea. Următorul pas este apelarea funcției **WiFi.softAP()** pentru a crea efectiv punctul de acces WiFi cu numele rețelei și parola specificate.

```
Serial.print("Setting AP (Access Point)...");  
WiFi.mode(WIFI_AP_STA);  
WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));  
WiFi.softAP(ssid, password);
```

Secvența de cod 6.4.2: *Setările pentru ESP8266 în modul AP*

Modulul ESP8266 poate fi utilizat ca un server web asincron pentru a oferi servicii web și a răspunde la cereri HTTP de la alte dispozitive. Pentru configurarea serverului se folosește biblioteca **ESPAsyncWebServer**. Inițializarea serverului web asincron se face utilizând clasa **AsyncWebServer**. Acesta va fi responsabil pentru gestionarea cererilor HTTP primite.

```
AsyncWebServer server(80);
```

Secvența de cod 6.4.3: *Inițializarea serverului web asincron*

Următorul pas îl reprezintă definirea rutelor și a funcțiilor de manipulare asociate acestora. Mai jos este definită ruta principală a serverului care trimite ca răspuns o pagină HTML.

```
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)  
{ request->send_P(200, "text/html", index_html); });
```

Secvența de cod 6.4.4: *Definirea rutei principale a server-ului*

Pagina HTML conține un meniu dropdown cu toate rețelele WiFi disponibile în apropiere din care se poate selecta rețeaua dorită, un câmp pentru introducerea parolei și un buton pentru trimiterea valorilor. Scanarea rețelelor se realizează folosind funcția **scanNetworks** disponibilă în biblioteca ESP8266. Aceasta returnează o listă de obiecte care conțin informații despre rețelele disponibile, cum ar fi numele rețelei (SSID-ul), puterea semnalului și securitatea rețelei.

```
server.on("/networks", HTTP_GET, [](AsyncWebServerRequest *request)  
{ String json = "[";  
  int n = WiFi.scanComplete();  
  if(n == -2){  
    WiFi.scanNetworks(true);  
  } else if(n){  
    for (int i = 0; i < n; ++i){  
      if(i) json += ",";  
      json += "{";  
      json += "\"rssi\":\""+String(WiFi.RSSI(i));  
      json += "\",\"ssid\":\""+WiFi.SSID(i)+"\"";  
    }  
  }  
  json += "]";  
  request->send(200, "text/json", json);  
}
```

```
        json += ",\"bssid\": \""+WiFi.BSSIDstr(i)+"\"";  
        json += ",\"channel\": "+String(WiFi.channel(i));  
        json += ",\"secure\": "+String(WiFi.encryptionType(i));  
        json += ",\"hidden\": "+String(WiFi.isHidden(i)?true:false);  
        json += "},";  
    }  
    if(WiFi.scanComplete() == -2){  
        WiFi.scanNetworks(true);  
    }  
}  
json += "]}";  
Serial.println(json);  
request->send(200, "application/json", json);  
json = String(); }
```

Secvența de cod 6.4.5: Definierea rutei care scanează toate rețelele disponibile

Metoda **begin** se folosește pentru a porni serverul web și a începe să asculte cereri HTTP.

6.5. CONECTAREA UNUI CLIENT MQTT

Pentru verificarea conexiunii unui client MQTT se folosește biblioteca **PubSubClient** pentru ESP8266 și secvența de cod de mai jos.

```
void reconnect()  
{  
    // Loop until the client is not connected  
    while (!client.connected())  
    {  
        Serial.print("Attempting MQTT connection...");  
        if (client.connect(sensorId, mqttUsername, mqttPassword))  
        {  
            // Change status to connected  
            mqttStatus = "connected";  
            Serial.println("connected");  
        }  
        else  
        {  
            // Change status to failed  
            mqttStatus = "failed";  
            Serial.print("failed, rc=");  
            Serial.print(client.state());  
            Serial.println(" try again in 1 second");  
            // Wait 1 second before retrying  
            delay(1000);  
        }  
    }  
}
```

Secvența de cod 6.5: Funcție pentru verificarea conexiunii unui client MQTT

În momentul în care conexiunea a reușit se schimbă statusul pentru MQTT și se începe transmiterea datelor citite de senzor.

6.6. ADĂUGAREA UNUI SENZOR

Procesul de adăugare a senzorilor este format din mai mulți pași. Toți pașii sunt definiți într-un wizard pentru a oferi claritate și ușurință în înțelegerea și gestionarea codului. De asemenea, prin împărțirea codului în etape, se poate separa și organiza logica specifică fiecărei etape.

```
<Wizard>
  <IpAddressStep1
    onClick={() => setEnterButton(true)}
    register={register}
  />
  <IpAddressStep2 ipAddress={getValues("ipAddress")} />
  <NameStep register={register} />
  <RoomStep setValue={setValue} />
  <WallTypeStep
    value={getValues("wallType")}
    register={register}
    onSubmit={onSubmit}
  />
  <CompleteStep selectedRoom={getValues("room")} onClose={onClose} />
</Wizard>
```

Secvența de cod 6.6.1: Componenta wizard cu toate etapele adăugării unui senzor

În secvența de mai sus, se pot observa cei șase pași necesari adăugării unui senzor în aplicație. Primul pas constă în introducerea adresei IP a senzorului, adresă care este obținută după conectarea modului ESP8266 la internet. Al doilea pas reprezintă atribuirea unui nume sugestiv senzorului, urmat de introducerea încăperii în care acesta va monitoriza valorile parametrilor. Ultimul pas constă în alegerea materialului din care sunt construiți pereții încăperii. Acest pas este foarte important deoarece unele materiale pot fi mai susceptibile la dezvoltarea mușgaiului din cauza capacității lor de a absorbi și reține umiditatea, creând astfel un mediu propice.

După introducerea tuturor informațiilor, se apelează funcția de mai jos care apelează metoda de POST pentru crearea unui nou senzor. Fiecare senzor este asociat unui utilizator, iar înainte de apelare se setează câmpul **userID** cu ID-ul utilizatorului autentificat.

```
const onSubmit = async () => {
  setValue("userId", localStorage.getItem("loggedUserId") ?? "");
  const postData = getValues();
  const result = await fetch(`${server}/sensors`, {
    method: "post",
    headers: {
```

```
"Content-Type": "application/json",  
"x-access-token": "token-value",  
},  
body: JSON.stringify(postData),  
});  
if (!result.ok) {  
  console.log(result);  
} else {  
  console.log("sensor added");  
}  
};
```

Secvența de cod 6.6.2: Funcție pentru apelarea metodei de POST

6.7. Scrierea și salvarea valorilor parametrilor

Citirea valorilor de la senzori se realizează atât timp cât aceștia sunt conectați la brokerul MQTT. Citire se realizează cu ajutorul metodele oferite de bibliotecă. În secvența de cod alăturată se poate observa că pentru a citi temperatura, umiditatea și presiunea, se utilizează metodele **readTemperature()**, **readHumidity()** și **readPressure()**.

```
// Read temperature, humidity and pressure from the BME280 sensor  
float temperature = bme.readTemperature();  
float humidity = bme.readHumidity();  
float pressure = bme.readPressure() / 100.0F;  
float altitude = bme.readAltitude(SEALEVELPRESSURE_HPA);
```

Secvența de cod 6.7.1: Citirea valorilor de la senzori

După citirea valorile de la senzor, se creează un obiect JSON care să reprezinte datele citite. Aceasta implică crearea unui obiect în care sunt atribuite valorile citite la cheile corespunzătoare, cum ar fi "temperatura", "umiditatea" și "presiunea". Pentru ca clientul MQTT să publice un mesaj pentru server este necesar ca acesta să fie șir de caractere. Astfel, este necesar un proces de serializare a obiectului JSON într-un șir de caractere.

Funcția **mqtt.publish()** este utilizată pentru a publica un mesaj pe un anumit subiect (topic) într-un broker MQTT. Topicul, în acest caz, este „**sensor/**” urmat de ID-ul senzorului.

```
char *topic;  
asprintf(&topic, "%s/%s", "sensor", sensorId);  
  
Serial.println("Publish message: " + msg);  
if (client.connect(sensorId, mqttUsername, mqttPassword))  
{  
  // Publish the message  
  client.publish(topic, msg.c_str());  
}
```

Secvența de cod 6.7.2: Publicarea unui mesaj

În momentul în care se primește un mesaj pe topicul la care serverul este abonat, se utilizează serviciul InfluxDB pentru a salva datele în baza de date. Metoda **getWriteAPI()** este o metodă disponibilă în cadrul bibliotecii InfluxDB pentru a obține o instanță a interfeței de scriere. Această interfață permite scrierea de date în baza de date InfluxDB. În secvența de mai jos, aceasta returnează o instanță asociată cu organizația **moldApp** și bucket-ul **sensors**.

Metoda **writePoint()** se utilizează pentru a salva un punct de date în baza de date. Punctul de date este creat folosind clasa **Point**, iar apoi sunt adăugate etichete (**tag**) și câmpuri (**floatField**,) cu valorile corespunzătoare. Metoda **flush()** se utilizează pentru a forța trimiterea tuturor punctelor de date către baza de date înainte de închiderea conexiunii. Aceasta asigură că toate datele în așteptare sunt trimise și actualizate în baza de date. De asemenea, se recomandă apelarea metodei **close()** după ce toate operațiile de scriere au fost finalizate pentru a elibera resursele.

```
async saveData(data: SensorValues, topic: string) {  
  const writeAPI = this.connection.getWriteApi('moldApp', 'sensors');  
  const point = new Point('myMeasurement')  
    .tag('sensor', topic)  
    .floatField('temperature', data.temperature)  
    .floatField('humidity', data.humidity);  
  writeAPI.writePoint(point);  
  writeAPI.flush();  
  writeAPI.close().then(() => console.log(point));  
}
```

Secvența de cod 6.7.3: Salvarea valorilor în baza de date

6.8. CALCULAREA INDICELUI DE MUȘGAI

Calcularea indicelui de mușgai se realizează cu ajutorul formulelor prezentate în capitolul 2.

```
export function calculateMmax(  
  A: number,  
  B: number,  
  C: number,  
  RHmin: number,  
  temperature: number,  
  relativeHumidity: number,  
) {  
  const RHcrit =  
    temperature < 20  
    ? -0.00267 * Math.pow(temperature, 3) +  
      0.16 * Math.pow(temperature, 2) -  
      3.13 * temperature +  
      100  
    : RHmin;
```

```
return (  
  A +  
  B * ((RHcrit - relativeHumidity) / (RHcrit - 100)) -  
  C * Math.pow((RHcrit - relativeHumidity) / (RHcrit - 100), 2)  
);  
}
```

Secvența de cod 6.8.1: *Funcție pentru calcularea indicelui de mușgai maxim*

Funcția de mai sus este folosită pentru calcularea indicelui de mușgai maxim. Aceasta primește 6 parametri, după cum urmează: **A**, **B**, **C**, **RHmin** sunt constante care corespund clasei de sensibilitate în care se încadrează materialul din care sunt construiți pereții încăperii în care se dorește monitorizarea valorilor, **temperature** și **relativeHumidity** sunt valorile curente ale temperaturii și umidității din încăperea respectivă.

Întrucât formula necesită cunoașterea valorii umidității critice, primul pas este calcularea acesteia. În cazul în care temperatura este mai mică de 20°C, valoarea umidității critice este egală cu valoarea lui RHmin. În caz contrar, se aplică formula (1). În final, funcția returnează valoarea lui Mmax folosind formula (2).

Mai jos, se poate observa funcția care calculează indicele de mușgai. Aceasta primește ca parametri indicii de mușgai maxim, calculat anterior, valoarea temperaturii și a umidității în momentul curent și încă două constante care corespund clasei de sensibilitate a materialului. **K1** poate lua valoarea lui **k1MLowerThan1** sau **k1MGreaterThan1** în funcție de valoarea curentă a lui M. Aceasta trebuie să fie mai mică decât 1 pentru a lua valoarea lui **k1MLowerThan1** și mai mare decât 1 în caz contrar. **K2** se calculează folosind formula (3). După calcularea acestor coeficienți, se poate calcula **deltaM** care reprezintă modificarea indicelui în timp. Dacă indicii curent este mai mic decât 0, atunci se setează indicii curent cu valoarea 0.

```
export function calculateMoldIndex(  
  mMax: number,  
  temperature: number,  
  relativeHumidity: number,  
  k1MLowerThan1: number,  
  k1MGreaterThan1: number,  
) {  
  const data: number[] = [];  
  let currentM = 0;  
  let previousM = 0;  
  let deltaM = 0;  
  let k1 = 0;  
  let k2 = 0;  
  for (let t = 0; t < 10; t = t + 1) {  
    k1 = currentM < 1 ? k1MLowerThan1 : k1MGreaterThan1;  
    k2 = Math.max(1 - Math.exp(2.3 * (currentM - mMax)), 0);  
    deltaM =  
      (k1 * k2) /  
      (7 *  
        Math.exp(  

```

```
-0.68 * Math.log(temperature) -  
13.9 * Math.log(relativeHumidity) +  
66.02,  
));  
currentM = previousM + deltaM;  
if (currentM < 0) {  
  currentM = 0;  
}  
previousM = currentM;  
data.push(currentM);  
}  
return data;  
}
```

Secvența de cod 6.8.2: Funcție pentru calcularea indicelui de mușgai

6.9. NOTIFICĂRI

Pentru partea de notificări a aplicației, a fost implementat un serviciu care gestionează logica de notificare. Acest serviciu conține un job care se execută în mod repetat la fiecare zece ore pentru a verifica valorile indicelui de mușgai. Pe baza acestor valori, salvează diferite tipuri de notificări în baza de date. Notificările sunt de trei tipuri: informative, de atenționare sau critice.

```
@Cron(CronExpression.EVERY_10_HOURS)  
async handleCron() {  
  const sensors = await this.sensorService.findAll();  
  const currentValues = sensors.map(async (sensor) => {  
    return {  
      parameters: await this.influxdbService.readData(sensor._id.toString()),  
      userId: sensor.userId,  
    };  
  });  
  Promise.all(currentValues).then((results) => {  
    results.map((result) => {  
      if (  
        result.parameters.moldIndex > 1 &&  
        result.parameters.moldIndex < 4  
      ) {  
        const date = new Date();  
        this.notificationService.create({  
          type: '1',  
          title: 'Hi there!',  
          message: 'Mold index is high. Take necessary precautions.',  
          viewed: false,  
          created_at: date,  
          userId: result.userId,  
        });  
      } else if (result.parameters.moldIndex >= 4) {  
        const date = new Date();
```

```
    this.notificationService.create({
      type: '2',
      title: 'Hi there!',
      message: 'Mold index is extremely high. Immediate action required!',
      viewed: false,
      created_at: date,
      userId: result.userId,
    });
  } else if (
    result.parameters.moldIndex >= 0 &&
    result.parameters.moldIndex <= 1
  ) {
    const date = new Date();
    this.notificationService.create({
      type: '3',
      title: 'Hi there!',
      message: 'Mold index is within acceptable range.',
      viewed: false,
      created_at: date,
      userId: result.userId,
    });
  }
});
});
```

Secvența de cod 6.9: Job programat pentru notificări

În secvența de mai sus, se poate observa că în NestJS se utilizează decoratorul **@Cron()** pentru a specifica programul de execuție al job-ului și funcția care trebuie apelată în momentele respective. În cazul de față, metoda verifică indicele de mușcăi pentru toți senzorii asociați unui anumit utilizator și generează o notificare în consecință. Fiecare notificare include un titlu, un mesaj sugestiv, tipul și momentul în care a fost creată.

Unul dintre cele mai importante avantaje ale unui job de notificări este capacitatea sa de a automatiza procesul de trimitere a notificărilor în cadrul aplicației. Acest lucru nu numai că elimină posibilitatea de a uita sau amâna notificările, dar asigură și consecvența în ceea ce privește momentul și modul în care notificările ajung la utilizatori.

TESTAREA ȘI UTILIZAREA APLICAȚIEI

7.1. PAGINA DE START

Prima pagină pe care utilizatorul o vede în momentul deschiderii aplicației este pagina de start. Pagina conține logo-ul aplicației, numele și motto-ul pe care se ghidează aplicația și anume: ***It's not mold until it's tested***. Numele **Moldsafe** sugerează că este o aplicație dedicată siguranței împotriva mușcăiului. Este o combinație între cuvintele „mold” (mușcăi) și „safe” (sigur).



Fig.7.1: Pagina de start

7.2. ÎNREGISTRAREA ÎN APLICAȚIE

În cazul în care utilizatorul nu are un cont creat în aplicație, acesta este nevoit să își creeze un cont. Crearea unui cont presupune introducerea unui nume, a unei adrese de email și a unei parole. Înainte de acest pas, se efectuează validări pentru câmpul email și pentru parolă. Email-ul trebuie să fie conțină caracterul @ care separă partea locală de partea de domeniu a adresei de email. Parola trebuie să conțină cel puțin 8 caractere, cel puțin o literă mare și cel puțin o cifră. Definirea unei reguli de validare a parolei este importantă pentru siguranța contului utilizatorului.

The figure displays three sequential screens of a mobile application for creating an account with 'moldsafe'.

- Screen 1 (Left):** The 'SIGN UP' screen. It features the 'moldsafe' logo and a message: 'Looks like you don't have an account. Let's create a new account for you.' Below this are three input fields: 'Name', 'Email *', and 'Password *'. At the bottom, there is a blue 'CREATE ACCOUNT' button and a '< BACK' link.
- Screen 2 (Middle):** The 'SIGN UP' screen showing validation errors. The 'Name' field is filled with 'Maria'. The 'Email *' field contains 'mar' and is highlighted with a red border, with the error message 'Invalid email address' displayed below it. The 'Password *' field is empty, and a red message below it states: 'Password should contains minimum 8 characters, at least one uppercase letter, one lowercase letter and one number'. The 'CREATE ACCOUNT' button and '< BACK' link are also present.
- Screen 3 (Right):** The 'SIGN UP' screen after successful validation. The 'Name' field is 'Maria', the 'Email *' field is 'maria@gmail.com', and the 'Password *' field contains a single dot. The 'CREATE ACCOUNT' button and '< BACK' link are at the bottom.

Fig.7.2: Ecranele pentru crearea unui cont

7.3. DASHBOARD-UL APLICAȚIEI

Pagina de dashboard a aplicației diferă în funcție de existența unui senzor pentru utilizatorul autentificat. În cazul în care nu există niciunul, singura acțiune posibilă este adăugarea unui senzor nou. Altfel, dashboard-ul va ilustra camerele în care au fost adăugați senzori și numele acestora.

The figure displays two versions of the 'My home' dashboard screen.

- Left Screen:** Shows 'My home' with the text 'No sensor available' and a blue 'ADD SENSOR' button. At the bottom is a navigation bar with 'Dashboard', a home icon, and a notification bell.
- Right Screen:** Shows 'My home' with a '+' icon in the top right. It features two sections: 'Scenes' with 'Bedroom 1' and 'Living room' (each with a toggle switch), and 'Sensors' with 'sensor 1' and 'sensor 2' (each with a toggle switch). At the bottom is a navigation bar with 'Dashboard', a home icon, and a notification bell.

Fig.7.3: Dashboard-ul aplicației

7.4. ADĂUGAREA UNUI SENZOR

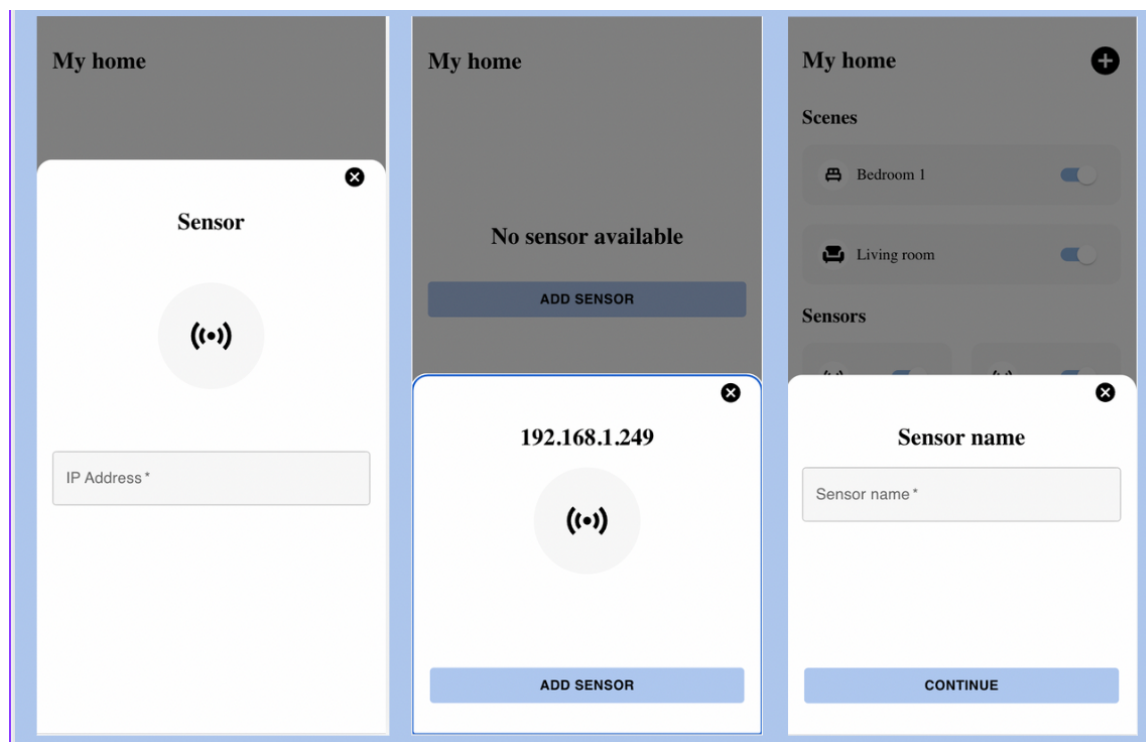


Fig.7.4.1: Primii pași ai adăugării unui senzor

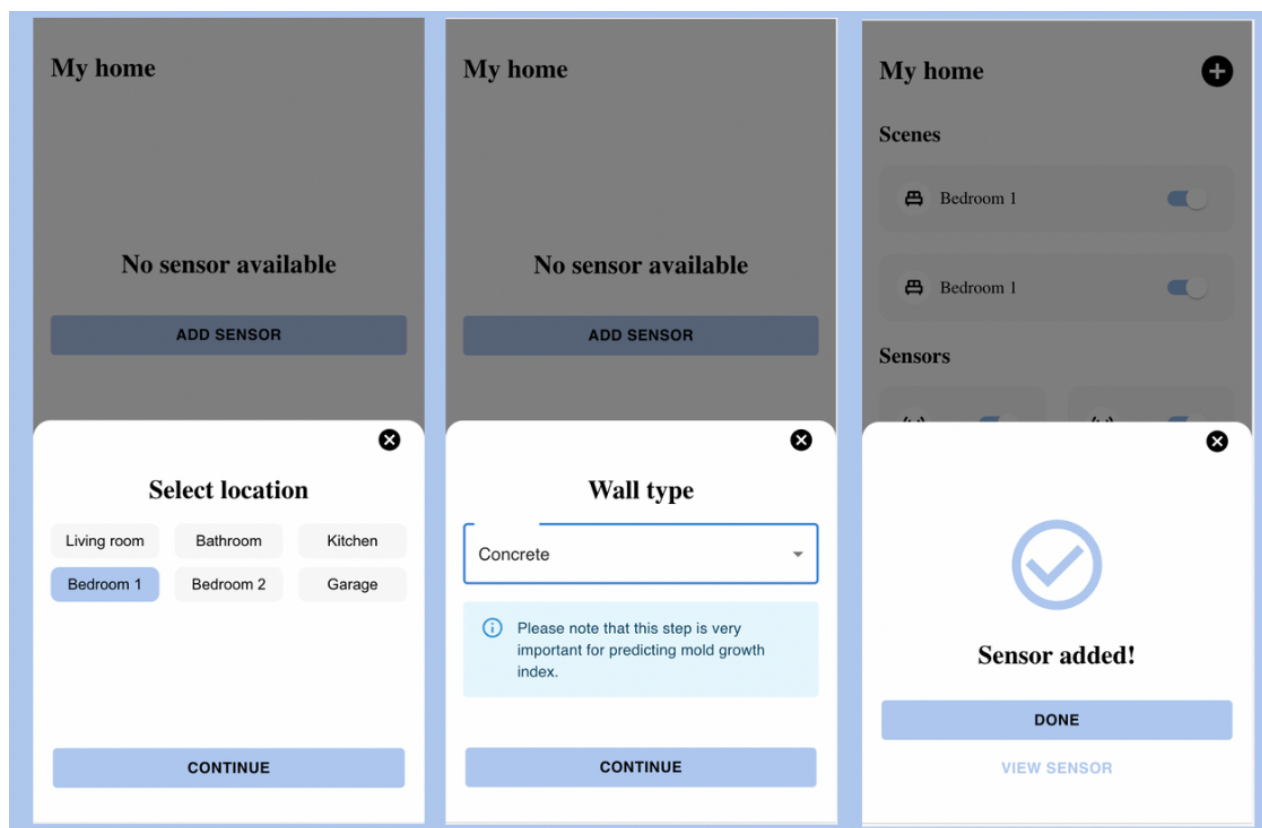


Fig.7.4.2: Următorii pași ai adăugării unui senzor

7.5. VIZUALIZAREA PARAMETRIILOR

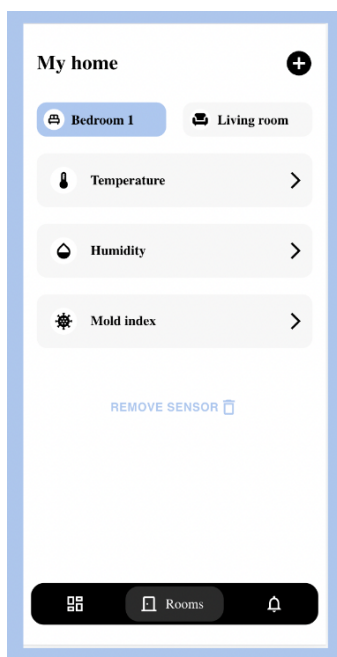


Fig.7.5.1: Overview al parametrilor măsurați

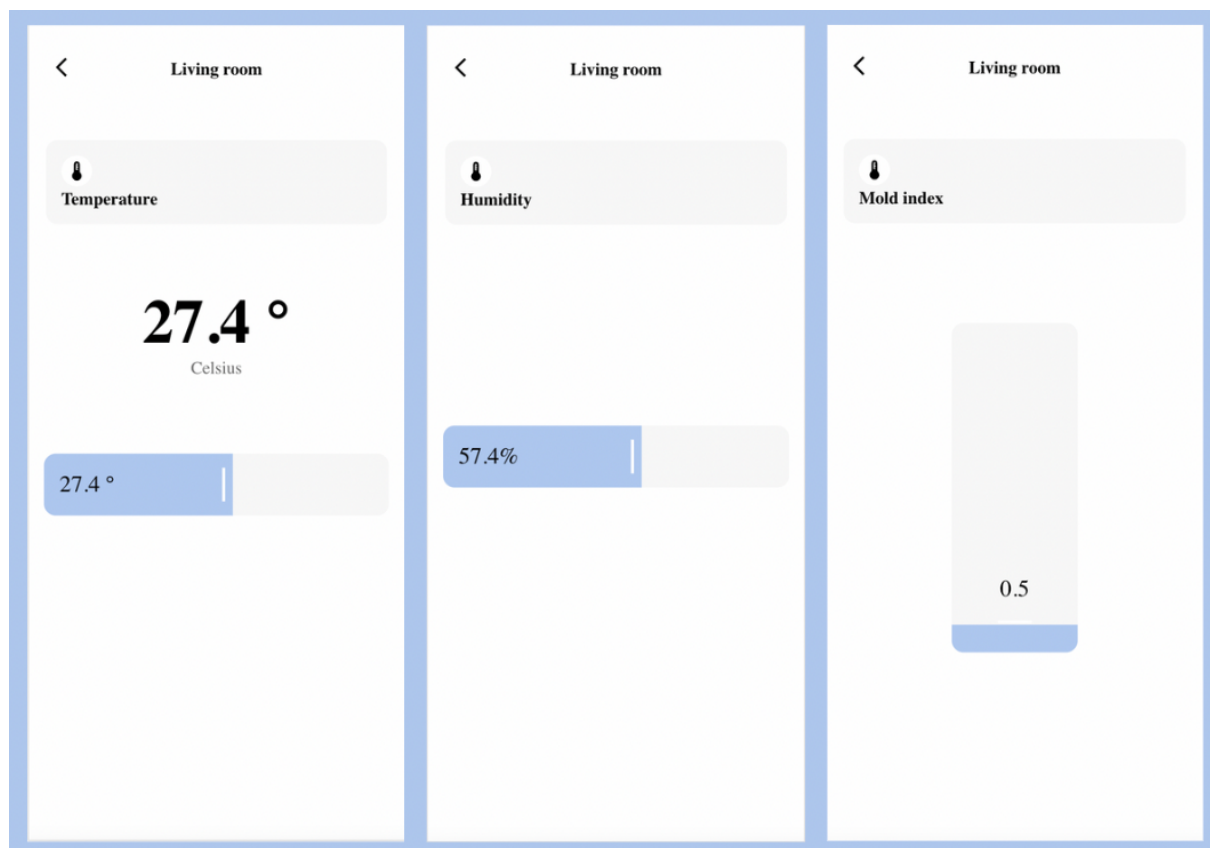


Fig.7.5.2: Ecranele pentru vizualizarea fiecărui parametru

7.6. NOTIFICĂRI

Pagina de notificări afișează o listă de notificări ordonată descendent în funcție de data creării acestora. Culoarea fiecărei notificări corespunde tipului acesteia, notificările informative apar cu negru, notificările de atenționare sau avertizare cu galben atunci când indicele de mușcăi a depășit un anumit prag, iar cele critice cu roșu. Mai mult, fiecare notificare are un meniu lateral cu câte două opțiuni: marcarea acesteia ca văzută și ștergerea.

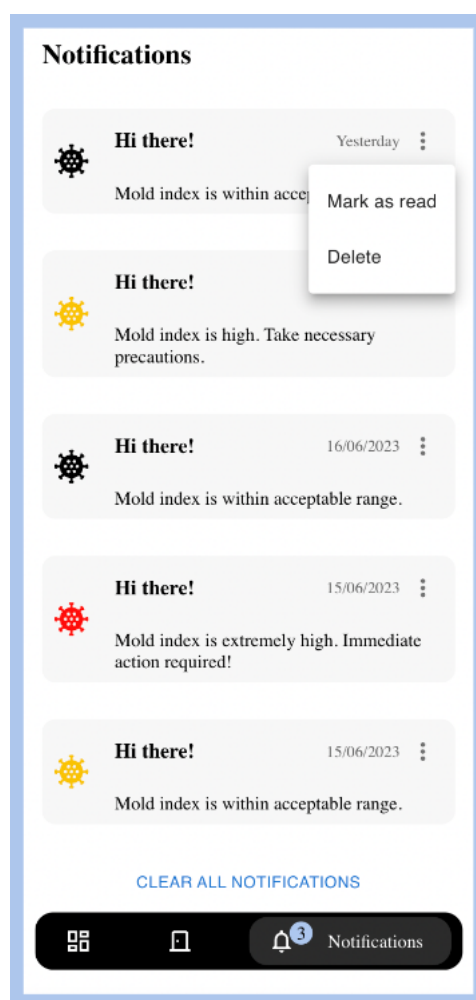


Fig.7.6: Pagina de notificări

7.7. TESTAREA MANUALĂ

Pentru testarea aplicației s-a folosit testarea manuală conform diagramei de mai jos (Fig.7.7). Primul pas a fost documentarea funcționalităților principale ale aplicației, urmat de planificarea și executarea testelor. De fiecare dată când rezultatele testelor nu au fost conform așteptărilor, au fost documentate erorile și s-au remediat. În final, s-a efectuat o reverificare a funcționalităților testate pentru a confirma că problemele identificate au fost rezolvate în mod corespunzător și că modificările nu au afectat alte părți ale aplicației. Acest

pas de reverificare a contribuit la asigurarea integrității și stabilității aplicației după aplicarea modificărilor.

Acest proces de testare manuală și documentare a fost aplicat de fiecare dată când s-a introdus o funcționalitate nouă în aplicație. Astfel, s-a asigurat că aplicația este fiabilă, funcțională și că răspunde în mod corespunzător cerințelor și așteptărilor utilizatorilor.

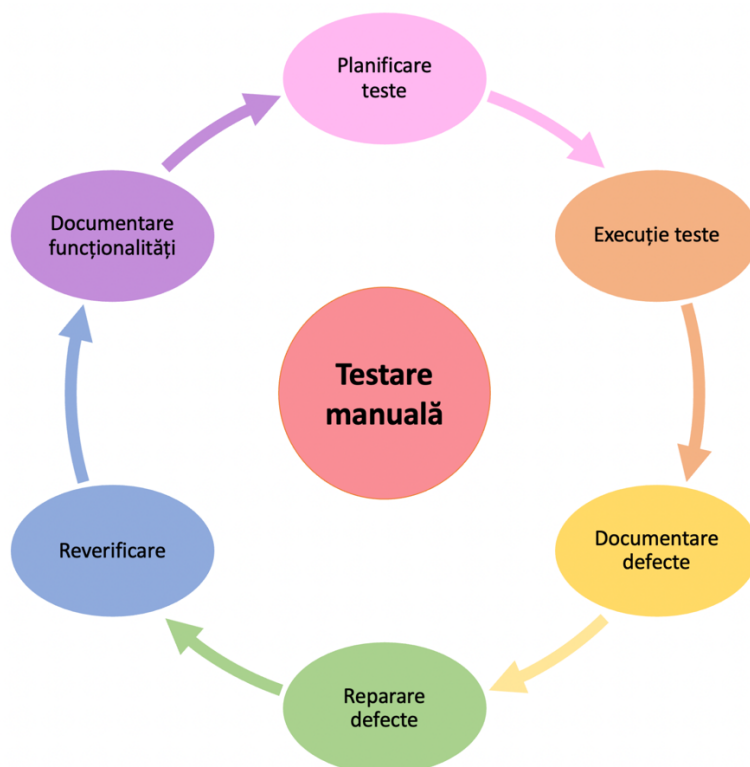


Fig.7.7: Procesul de testare

5. CONCLUZII

Aplicația **Moldsafe** pentru monitorizarea temperaturii, umidității și mucegaiului are o importanță semnificativă în menținerea unui mediu sănătos și sigur în interiorul încăperilor. Prin colectarea datelor în timp real, aplicația oferă utilizatorilor informații esențiale despre nivelurile critice ale acestor parametri. Această informație permite utilizatorilor să identifice rapid și să acționeze asupra situațiilor care ar putea favoriza dezvoltarea mucegaiului, precum fluctuațiile extreme de temperatură sau nivelurile ridicate de umiditate.

Prin intermediul unei interfețe intuitive și prietenoase, **Moldsafe** permite vizualizarea și analizarea în timp real a datelor colectate, oferind un overview clar al condițiilor din fiecare încăpere. Utilizatorii pot identifica rapid încăperile cu valori anormale ale temperaturii sau umidității, luând măsurile necesare pentru a le corecta și a preveni apariția mucegaiului.

Folosind tehnologii moderne și o comunicare asincronă, aplicația oferă o soluție scalabilă și permite adăugarea unui număr mare de senzori. Datele colectate pot fi stocate în siguranță într-o bază de date, permițând utilizatorilor să acceseze și să analizeze informațiile în orice moment și de oriunde. De asemenea, utilizatorii pot primi notificări în cazul în care valorile critice depășesc limitele setate.

Un aspect important al aplicației este că nu se limitează doar la dispozitivele folosite. Aplicația poate colecta date de la orice dispozitiv care se conectează la brokerul MQTT configurat. Pentru colectarea datelor de temperatură și umiditate, aplicația **Moldsafe** folosește senzorii BME280 care oferă măsurători precise și stabile. Acești senzori sunt conectați la module ESP8266, care acționează ca noduri de rețea și transmit datele către serverul central. Serverul central, dezvoltat folosind tehnologii moderne procesează și analizează datele primite de la senzori. Pentru a asigura o experiență de utilizare optimă și o conectivitate stabilă, aplicația folosește protocolul MQTT (Message Queuing Telemetry Transport). Acesta permite transmiterea și recepționarea datelor în timp real între senzori, server și aplicația mobilă.

În ceea ce privește stocarea datelor, se utilizează o bază de date performantă precum InfluxDB, care permite stocarea și gestionarea eficientă a informațiilor colectate pe bază temporală. Aceste informații pot fi apoi vizualizate într-o interfață intuitivă, care oferă o imagine de ansamblu a valorilor în diferite încăperi.

Un aspect care ar fi putut fi abordat diferit în aplicația **Moldsafe** ar fi dezvoltarea și implementarea unui algoritm mai eficient de predicție a riscului de formare a mucegaiului. Acest algoritm ar putea fi optimizat prin utilizarea unor tehnici avansate de analiză a datelor și de învățare automată ar putea contribui la obținerea unor predicții mai exacte, oferind utilizatorilor informații mai detaliate și utile pentru prevenirea și gestionarea riscului de mucegai în încăperi.

În concluzie, aplicația **Moldsafe** reprezintă o soluție indispensabilă pentru monitorizarea și prevenirea mucegaiului în interiorul încăperilor. Prin furnizarea de informații precise și oportune, utilizatorii pot lua măsuri preventive și corective în timp util, asigurând un mediu sănătos, sigur și confortabil.

6. DIRECȚII VIITOARE

În urma implementării funcționalităților de monitorizare a temperaturii, umidității și mucegaiului, s-a obținut o soluție valoroasă pentru utilizatori. Numai că, suntem conștienți că tehnologia și nevoile utilizatorilor evoluează constant, iar aplicația trebuie să rămână actuală și competitivă. Prin urmare, acest capitol explorează posibilitățile de îmbunătățire și extindere ale aplicației.

Una dintre direcțiile viitoare este reprezentată de integrarea cu un serviciu OpenID Connect (OIDC). Prin adăugarea funcționalității de autentificare și control al accesului bazat pe standardul OIDC, aplicația devine mai securizată și mai ușor de gestionat pentru utilizatori. Aceasta oferă o experiență de autentificare unificată și ar putea elimina necesitatea de a crea și menține propriul sistem de autentificare. Integrarea se poate realiza cu servicii de identitate existente, cum ar fi Google, Facebook sau Microsoft. De asemenea, aceasta facilitează implementarea unor caracteristici de securitate avansate, cum ar fi autentificarea în doi pași sau autentificarea bazată pe token-uri. Prin urmare, aplicația devine mai robustă în fața atacurilor de securitate și oferă utilizatorilor un grad mai mare de protecție a datelor personale.

Tot ca direcție viitoare, poate fi considerată integrarea altor senzori în aplicația de față pentru a extinde funcționalitățile de monitorizare. Aceasta ar include adăugarea și configurarea de senzori suplimentari, cum ar fi senzori de lumină, senzori de zgomot, senzori de vânt sau senzori de calitate a aerului. Acești senzori ar permite utilizatorilor noștri să obțină o imagine mai completă și detaliată a mediului din încăperi. Integrarea acestor senzori adiționali ar oferi o soluție mai complexă și versatilă, care să satisfacă și alte nevoi ale utilizatorilor. De exemplu, cu ajutorul unui senzor de vânt integrat, aplicația ar putea furniza date actualizate despre vânt, permițând utilizatorilor să ia măsuri adecvate, precum închiderea ferestrelor sau ajustarea activităților exterioare în funcție de intensitatea și direcția vântului.

Network discovering este o altă direcție viitoare care simplifică procesul de on-board al senzorilor. Aceasta constă în integrarea unei funcționalități în aplicație care poate detecta și identifica adresa IP atribuită modului în rețea. Prin intermediul acestei funcționalități, aplicația ar căuta automat modulul ESP8266 în rețea și ar identifica adresa IP asociată acestuia. Beneficiile aduse de network discovering includ ușurința în configurarea inițială a aplicației, reducerea timpului și efortului necesar pentru introducerea manuală a informațiilor despre rețea și eliminarea erorilor umane asociate cu acest proces. Experiența utilizatorului s-ar îmbunătăți și ar face ca procesul de on-board să fie mai eficient și mai ușor de parcurs pentru toți utilizatorii. Totuși, această funcționalitate este complexă și implică utilizarea protocoalelor de rețea, manipularea pachetelor de date și gestionarea comunicării între aplicație și modulul ESP8266. De asemenea, este importantă documentarea adecvată a protocoalelor și a bibliotecilor folosite, precum și testarea riguroasă a funcționalității pentru a asigura o conexiune stabilă și fiabilă între aplicație și modulul ESP8266.

BIBLIOGRAFIE

- [1]. Desai, S.; Manapragada, N.V.S.K.; Shukla, A.K.; Pignatta, G. Mould-Growth Study in Building Materials Exposed to Warm and Humid Climate Using Heat and Mass Transfer (HAMT) EnergyPlus Simulation Method. *Sustainability*, 2022, 14, 8292.
- [2]. OJANEN, Tuomo, et al. Mold growth modeling of building structures using sensitivity classes of materials. In: *11th International Conference on Thermal Performance of the Exterior Envelopes of Whole Buildings, Buildings XI*. 2010.
- [3]. ***<https://www.airthings.com/en-gb/wave-mini> (accesare mai 2023)
- [4]. ***<https://foobot.io/guides/air-quality-monitor.php> (accesare mai 2023)
- [5]. PARIHAR, Yogendra Singh, et al. Internet of things and nodemcu. *journal of emerging technologies and innovative research*, 2019, 6.6: 1085.
- [6]. ***<https://lastminuteengineers.com/esp8266-pinout-reference/> (accesare mai 2023)
- [7]. AZIZ, Dlnya Abdulahad. Webserver based smart monitoring system using ESP8266 node MCU module. *International Journal of Scientific & Engineering Research*, 2018, 9.6: 801-808.
- [8]. HEMMANUR, Karthik. Inter-Integrated Circuit (I2C). *Electrical Computer Engineering*, 2009.
- [9]. ***<https://lastminuteengineers.com/bme280-arduino-tutorial/> (accesare mai 2023)
- [10]. SONI, Dipa; MAKWANA, Ashwin. A survey on mqtt: a protocol of internet of things (iot). In: *International conference on telecommunication, power analysis and computing techniques (ICTPACT-2017)*. 2017. p. 173-177.
- [11]. ***<https://blog.mallow-tech.com/2018/03/mqtt-protocol-for-iot-a-brief-introduction/> (accesare mai 2023)
- [12]. MOSQUITTO, Eclipse. Eclipse mosquitto. *Eclipse Mosquitto*, 2018.
- [13]. PHAM, Anh Duc. *Developing back-end of a web application with NestJS framework: Case: Integrify Oy's student management system*, 2020.
- [14]. Documentație MQTT - <https://docs.nestjs.com/microservices/mqtt> (accesare mai 2023)
- [15]. Documentație Mongoose - <https://docs.nestjs.com/techniques/mongodb> (accesare mai 2023)
- [16]. DAO, Phong. SCALABLE REACT APPLICATION WITH REDUX AND TYPESCRIPT. 2022.
- [17]. ***<https://isamatov.com/react-component-composition/> (accesare mai 2023)
- [18]. MARTIN, Robert Cecil. *Agile software development: principles, patterns, and practices*. Prentice Hall PTR, 2003.
- [19]. Documentație React Use Form - <https://react-hook-form.com/> (accesare mai 2023)
- [20]. Documentație SWR - <https://swr.vercel.app/> (accesare mai 2023)
- [21]. *** <https://mui.com/material-ui/material-icons/> (accesare mai 2023)
- [22]. *** <https://ostechnix.com/introduction-to-docker-compose/> (accesare mai 2023)

LISTA FIGURILOR

Capitolul 2

Fig. 2.1.1: Valorile indiciile de creștere a mușcăiului.....	7
Fig. 2.1.2: Clasele de sensibilitate pentru diferite material.....	7
Fig. 2.1.3: Valorile parametrilor pentru clasele de sensibilitate	9
Fig 2.2.1: Dispozitivul Wave Mini.....	10
Fig.2.2.2: Dispozitivul Foobot.....	10

Capitolul 3

Fig. 3.1: Modulul ESP8266 NodeMCU.....	12
Fig. 3.2: Pinii senzorului BME280.....	14

Capitolul 4

Fig. 4.1: Arhitectura MQTT.....	15
Fig.4.4.1: Exemplu de funcționare a controllerelor în Nest	17
Fig.4.4.2: Exemplu de funcționare a modulelor în Nest	18
Fig. 4.5.1: React composition	20
Fig.4.5.2: Modul de căutare al iconițelor.....	22
Fig. 4.8: Modul de funcționare al Docker Compose.....	26
Fig. 4.10: Codul înainte și după formatarea cu Prettier.....	27

Capitolul 5

Fig. 5.1: Arhitectura sistemului.....	28
Fig.5.2.1: Conectarea modului ESP8266 și a senzorului BME280.....	29
Fig.5.2.2: Structura finală a componentelor.....	30
Fig.5.3.1: Diagrama modulelor.....	31
Fig.5.3.2: Diagrama de use case-uri.....	32

Capitolul 7

Fig.7.1: Pagina de start.....	45
Fig.7.2: Ecranele pentru crearea unui cont.....	46
Fig.7.3: Dashboard-ul aplicației.....	46
Fig.7.4.1: Primii pași ai adăugării unui senzor.....	47
Fig.7.4.2: Următorii pași ai adăugării unui senzor.....	47
Fig.7.5.1: Overview al parametrilor măsurați.....	48
Fig.7.5.2: Ecranele pentru vizualizarea fiecărui parametru.....	48

Fig.7.6: <i>Pagina de notificări</i>	49
Fig.7.7: <i>Procesul de testare</i>	50

LISTA SECVENȚELOR DE COD

Secvența de cod 6.1: Configurarea broker-ului MQTT.....	33
Secvența de cod 6.2: Configurare development container.....	34
Secvența de cod 6.3.1: Inițializarea metodelor folosite.....	35
Secvența de cod 6.3.2: Înregistrare și validare câmp.....	35
Secvența de cod 6.3.3: Metodă pentru crearea unui utilizator.....	36
Secvența de cod 6.3.4: Endpoint de tip POST pentru adăugarea unui utilizator.....	36
Secvența de cod 6.4.1: Parametrii pentru ESP8266 în modul AP.....	36
Secvența de cod 6.4.2: Setările pentru ESP8266 în modul AP.....	37
Secvența de cod 6.4.3: Inițializarea serverului web asincron.....	37
Secvența de cod 6.4.4: Definirea rutei principale a server-ului.....	37
Secvența de cod 6.4.5: Definirea rutei care scanează toate rețelele disponibile.....	38
Secvența de cod 6.5: Funcție pentru verificarea conexiunii unui client MQTT	38
Secvența de cod 6.6.1: Componenta wizard cu toate etapele adăugării unui sensor.....	39
Secvența de cod 6.6.2: Funcție pentru apelarea metodei de POST.....	40
Secvența de cod 6.7.1: Citirea valorilor de la senzori.....	40
Secvența de cod 6.7.2: Publicarea unui mesaj.....	41
Secvența de cod 6.7.3: Salvarea valorilor în baza de date.....	41
Secvența de cod 6.8.1: Funcție pentru calcularea indicelui de mușcăi maxim.....	42
Secvența de cod 6.8.2: Funcție pentru calcularea indicelui de mușcăi.....	43
Secvența de cod 6.9: Job programat pentru notificări.....	44

LISTA TABELELOR

Tabelul 1: <i>Analiză funcționalități</i>	11
---	----

LISTA FORMULELOR

Formula (1).....	8
Formula (2).....	8
Formula (3).....	9
Formula (4).....	9
Formula (5).....	9

**DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR***

Subsemnatul AVRAMOV AMALIA-GORDANA

legitimată cu C.I. seria TŽ nr. 462290

CNP 6000819350051

autorul lucrării MOLDSAFE - APLICAȚIE PENTRU PREDICȚIA
APARIȚIEI MUCEGAIULUI

elaborată în vederea susținerii examenului de finalizare a studiilor de LICENȚĂ organizat de către Facultatea AUTOMATICĂ ȘI CALCULATOARE din cadrul Universității Politehnica Timișoara, sesiunea Iunie 2023 a anului universitar 2022-2023, coordonator S.L. DR. ING. DANIELA STĂNESCU, luând în considerare art. 34 din Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

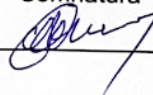
Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

24.06.2023

Semnătura



*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

¹ Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA